

Table of Contents

Ruby: Seu Portal para o Submundo Digital	2
Iniciando sua Jornada no Submundo Ruby	5
Equipando seu Arsenal Ruby	9
Primeiros Passos com Ruby	13
Sintaxe Básica do Ruby	18
Tipos de Dados em Ruby	25
Strings em Ruby	31
Tipos Numéricos em Ruby	36
Arrays e Hashes em Ruby	41
Ranges em Ruby	46
Entrada e Saída em Ruby	51
Entrada e Saída Básica em Ruby	54
Trabalhando com Arquivos em Ruby	59
Estruturas de Controle em Ruby	65
Operadores em Ruby	72
Condicionais em Ruby	77
Loops em Ruby	83
Blocos e Iteradores em Ruby	90
Conceitos Básicos de Blocos	96
Iteradores em Ruby	101
Enumerators em Ruby	106
Closures em Ruby	111
Programação Orientada a Objetos em Ruby	116
Classes e Objetos em Ruby	124
Herança em Ruby	132
Encapsulamento em Ruby	140
Composição em Ruby	148

Ruby: Seu Portal para o Submundo Digital

```
88888888b.      888
888  Y88b      888
888   888      888
888  d88P888  888888888b. 888 888
888888888P" 888 8888888 "88b888 888
888 T88b 888 8888888 8888888 888
888 T88b Y88b 8888888 d88PY88b 888
888 T88b "Y8888888888P" "Y888888
                        888
                        Y8b d88P
                        "Y88P"
```

 "Em um mundo onde código é lei, Ruby é a arma do rebelde elegante."

- *Yukihiro "Matz" Matsumoto, enquanto hackeava a Matrix*

Bem-vindo ao Submundo, Netrunner

Então você decidiu mergulhar no mundo do Ruby? Boa escolha, recruta. Em 1995, enquanto as megacorps ainda tentavam decidir se a internet era uma moda passageira, Matz estava craftando algo revolucionário: uma linguagem que trataria você como um ser humano, não como um processador glorificado.

Por que escolher a pílula Ruby?

Tudo é um Objeto (Não, sério, TUDO mesmo)

```
# Até números têm métodos. Mind = Blown
-42.abs.to_s.reverse #=> "24"
```

Sintaxe que não parece código alienígena

```
def hackear_mainframe
  puts "Acesso garantido" if sistema.vulneravel?
  sistema.explodir unless usuario.admin?
end
```

Flexibilidade de um contorcionista digital

Ruby te deixa resolver problemas de múltiplas formas. É como ter várias rotas de fuga no seu próximo heist digital.

O que você vai desbloquear neste tutorial

-  **Tutorial Básico:** Do "Hello, World" ao "Já hackeei a NASA"
-  **Tipos de Dados:** Strings, números e outras armas do seu arsenal
-  **I/O e Arquivos:** Porque dados são a nova moeda
-  **Loops e Iteradores:** Para quando você precisa fazer algo repetidamente (como tentar senhas)
-  **OOP:** Onde objetos são mais que apenas dados

Pré-requisitos

- Um terminal (quanto mais neon, melhor)
- Conhecimento básico de programação (se você já hackeou uma calculadora, serve)
- Um cérebro funcional (café não incluso)
- Vontade de dominar o mundo (opcional, mas recomendado)

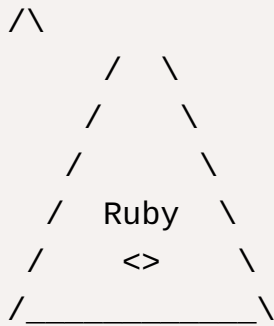
Convenções deste Manual de Sobrevivência

```
# Código aparecerá assim, com comentários sarcásticos inclusos
def exemplo_super_util
  puts "Você ainda está lendo os comentários?"
end
```

⚠ ALERTA DE SEGURANÇA: Blocos como este contêm informações cruciais para sua sobrevivência no submundo do código.

Próxima Missão

Prepare seu ambiente de desenvolvimento no Primeiros Passos ([Primeiros Passos com Ruby](#)). E não, "mkdir pasta_secreta" não conta como preparação de ambiente.



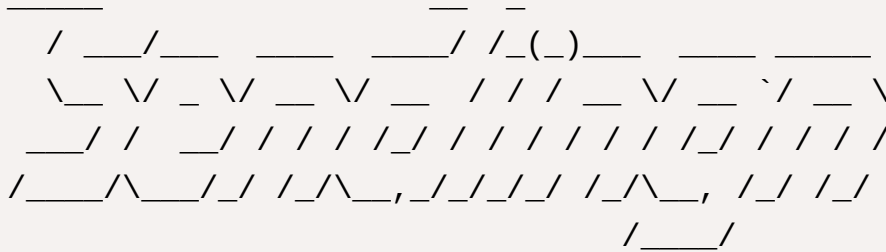
⚠ "Código limpo é para os fracos. Código elegante é para os fortes."

- *Algum programador Ruby às 3 da manhã*

See also

Quartel General do Ruby (<https://www.ruby-lang.org>)
Documentação para Netrunners (<https://ruby-doc.org>)

Iniciando sua Jornada no Submundo Ruby



"Todo hacker tem que começar de algum lugar. Geralmente é quebrando algo."

- *Manual do Netrunner Iniciante*



Configurando seu Deck Cybernético

Antes de hackear a Matrix, você precisa preparar seu ambiente. Aqui está seu checklist de sobrevivência:

Hardware Necessário

- Um computador (duh!)
- Teclado funcional (RGB opcional, mas recomendado para +10 de swag)
- Conexão com a internet (quanto mais rápida, menos tempo olhando para barras de progresso)

Software Essencial

- Terminal (seu portal para o submundo)
- Editor de texto (Vim para os masoquistas, VSCode para os mortais)

- Ruby (obviamente, né?)
- Git (porque você não é um animal para não versionar código)

Objetivos desta Missão

OBJETIVOS DA MISSÃO:

- ☐ Instalar Ruby
- ☐ Configurar Ambiente
- ☐ Primeiro Hack
- ☐ Não quebrar nada

Injetando Ruby no Sistema

Dependendo do seu OS (Sistema Operacional, para os novatos), o processo pode ser:

Windows (aka "O Sistema Corporativo")

```
# 1. Baixe o instalador em ruby-lang.org
# 2. Next -> Next -> Next -> Finish
# 3. Reze para o deus do PATH
```

Linux (aka "O Sistema dos Hackers")

```
# Ubuntu/Debian
sudo apt-get install ruby-full

# Arch (para os verdadeiros cyberpunks)
sudo pacman -S ruby
```

macOS (aka "O Sistema dos Ricaços")

```
# Usando Homebrew (porque você é cool)
brew install ruby
```

```
# Ou use o rbenv, se você gosta de complicar
rbenv install 3.2.0
```

Testando o Setup

Abra seu terminal (ou Matrix prompt) e digite:

```
ruby -v
```

Se você ver algo como `ruby 3.2.0p0`, parabéns! Você está oficialmente no jogo.

Seu Primeiro Hack

Vamos fazer algo mais interessante que "Hello World". Abra seu editor e crie:

```
# hack_the_planet.rb
def iniciar_sequencia_hack
  print "Iniciando sequência de hack"
  3.times { sleep 0.5; print "." }
  puts "\nAcesso garantido! A Matrix é sua!"
end

iniciar_sequencia_hack
```

Próximos Passos

Agora que você tem seu ambiente configurado, é hora de:

1. Instalar as Ferramentas ([Equipando seu Arsenal Ruby](#)) - Porque todo hacker precisa de seu arsenal
2. Primeiros Passos ([Primeiros Passos com Ruby](#)) - Aprenda a andar antes de correr pela Matrix
3. Sintaxe Básica ([Sintaxe Básica do Ruby](#)) - As regras do jogo (que foram feitas para serem quebradas)

 **DICA DE SOBREVIVÊNCIA:** Mantenha café por perto. Muito café.



\\|/

```
-----  
|  CAFFEINE  |  
| INJECTION  |  
|  SYSTEM   |  
-----
```



Avisos Importantes

- Não tente hackear a NASA (ainda)
- Sim, você vai quebrar coisas
- Não, ctrl+z não resolve tudo
- Stack Overflow será seu melhor amigo

 "Se não está funcionando, tente `sudo`. Se ainda não funcionar, tente café."

- *Sabedoria antiga dos mestres Ruby*

See also

RVM - Para os indecisivos que querem todas as versões (<https://rvm.io>)
Bundler - Seu gerenciador de dependências favorito (<https://bundler.io>)

Equipando seu Arsenal Ruby



"Um desenvolvedor sem suas ferramentas é como um samurai sem katana - precisa do equipamento certo para fazer um bom trabalho."

- *Manual do Desenvolvedor Ruby*



Kit de Desenvolvimento Essencial

Editor de Código (Seu Ambiente Principal)

♥ VSCode (Editor Moderno e Versátil)

```
# Windows/macOS
# Baixe em code.visualstudio.com

# Linux
sudo snap install code # Ubuntu
sudo pacman -S code # Arch Linux
```

Extensões Essenciais:

- Ruby
- Endwise (Fechamento automático de blocos)
- Ruby Solargraph (Autocompletar inteligente)

Vim/Neovim (Editor Avançado)

```
# Instalação
sudo apt install neovim # Ubuntu
sudo pacman -S neovim # Arch Linux
brew install neovim # macOS
```

Gerenciadores de Versão

RVM (Ruby Version Manager)

```
# Instalação básica
gpg --keyserver keyserver.ubuntu.com --recv-keys
409B6B1796C275462A1703113804BB82D39DC0E3
7D2BAF1CF37B13E2069D6956105BD0E739499BDB

\curl -sSL https://get.rvm.io | bash
```

rbenv (Alternativa Leve)

```
# macOS
brew install rbenv ruby-build

# Linux
git clone https://github.com/rbenv/rbenv.git ~/.rbenv
git clone https://github.com/rbenv/ruby-build.git
~/.rbenv/plugins/ruby-build
```

Gemas Essenciais

```
# Gemas fundamentais para desenvolvimento
gem install bundler # Gerenciador de dependências
gem install rails # Framework web completo
gem install pry # Depurador interativo
gem install rubocop # Analisador de código
```

Testando sua Instalação

```
# Crie um arquivo test_setup.rb
require 'bundler'
require 'rails'
require 'pry'

puts "✓ Bundler #{Bundler::VERSION}"
puts "✓ Rails #{Rails::VERSION::STRING}"
puts "✓ Ruby #{RUBY_VERSION}"
puts "\nInstalação concluída com sucesso!"
```

Configuração do Terminal

Oh My Zsh (Personalização do Terminal)

```
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

Temas Recomendados:

- Powerlevel10k (Interface moderna)
- Agnoster (Visual elegante)
- Dracula (Tema escuro profissional)



Resolução de Problemas Comuns

Problema	Solução Padrão	Solução Alternativa
"Comando não encontrado"	Adicionar ao PATH	Reiniciar o terminal
Permissão negada	<code>chmod +x</code>	<code>sudo !!</code>
Gem não instala	Verificar documentação	Consultar StackOverflow
Terminal não responde	Ctrl+C	Fechar e reabrir terminal



Checklist Final

✓ Ruby instalado	
✓ Editor configurado	
✓ Gemas básicas	
✓ Terminal personalizado	
✓ Ambiente pronto	



DICA PROFISSIONAL: Mantenha um terminal aberto com `tail -f log/development.log` para monitoramento em tempo real do seu aplicativo.

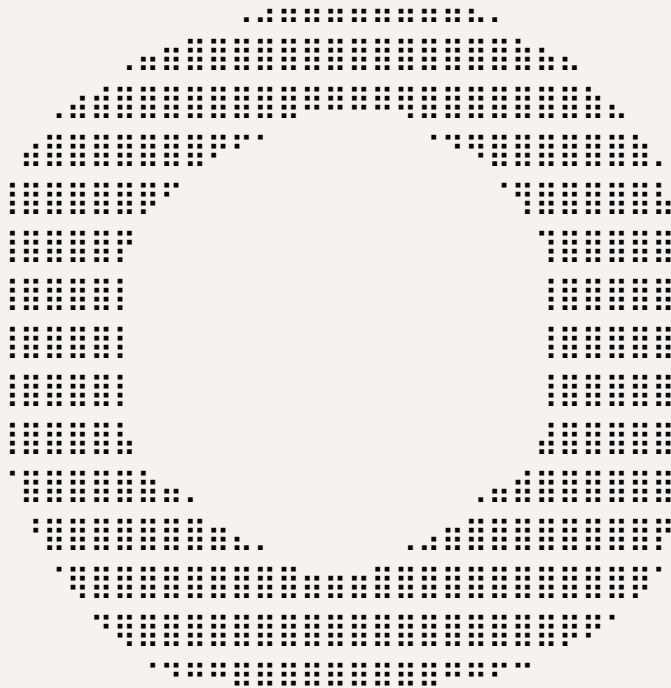
See also

RVM - Guia Completo (<https://rvm.io/rvm/install>)

rbenv - Documentação (<https://github.com/rbenv/rbenv#installation>)

VSCode + Ruby (<https://code.visualstudio.com/docs/languages/ruby>)

Primeiros Passos com Ruby



Iniciando sua Jornada em Ruby

Bem-vindo ao seu primeiro contato prático com Ruby! Vamos começar com os fundamentos essenciais que todo desenvolvedor Ruby precisa conhecer.

O Console Interativo (IRB)

O IRB (Interactive Ruby) é sua primeira ferramenta de experimentação:

```
# Abra o terminal e digite
irb

# Experimente algumas operações básicas
2 + 2           #=> 4
"Ruby".upcase   #=> "RUBY"
[1, 2, 3].map { |n| n * 2 }  #=> [2, 4, 6]
```

Seu Primeiro Programa Ruby

Crie um arquivo chamado `hello.rb`:

```
# hello.rb
def saudacao(nome)
  "Olá, #{nome}! Bem-vindo ao mundo Ruby!"
end

# Testando nossa função
puts saudacao("Desenvolvedor")
```

Execute seu programa:

```
ruby hello.rb
```

Estruturas Básicas

```
# Variáveis
nome = "Ruby"
idade = 30
linguagens = ["Ruby", "Python", "JavaScript"]

# Condicionais
if idade >= 18
  puts "Maior de idade"
else
  puts "Menor de idade"
end

# Loops
linguagens.each do |lang|
  puts "Eu conheço #{lang}"
end
```

Conceitos Fundamentais

1. Tudo é um Objeto

```
# Até números são objetos
42.class          #=> Integer
42.to_s           #=> "42"
42.methods        #=> [lista de métodos disponíveis]
```

2. Blocos são Fundamentais

```
# Diferentes formas de usar blocos
3.times { puts "Ruby!" }

3.times do |i|
  puts "Contagem: #{i + 1}"
end
```

3. Convenções Importantes

```
# Nomes de variáveis e métodos: snake_case
primeiro_nome = "Ruby"
def calcular_total
  # código aqui
end

# Nomes de classes: CamelCase
class MinhaClasse
  # código aqui
end
```

Exercícios Práticos

1. Calculadora Simples

```
def calculadora(n1, n2, operacao)
  case operacao
  when "+"
    n1 + n2
  end
end
```

```

when "-"
  n1 - n2
when "*"
  n1 * n2
when "/"
  n1.to_f / n2
else
  "Operação inválida"
end
end

```

2. Lista de Tarefas

```

class ListaTarefas
  def initialize
    @tarefas = []
  end

  def adicionar(tarefa)
    @tarefas << tarefa
    puts "Tarefa adicionada: #{tarefa}"
  end

  def listar
    @tarefas.each_with_index do |tarefa, index|
      puts "#{index + 1}. #{tarefa}"
    end
  end
end

```

Próximos Passos

Após dominar estes conceitos básicos, você estará pronto para explorar:

1. Sintaxe Básica ([Sintaxe Básica do Ruby](#)) - Aprofunde-se na linguagem

2. Tipos de Dados ([Tipos de Dados em Ruby](#)) - Conheça as estruturas fundamentais
3. Controle de Fluxo ([Estruturas de Controle em Ruby](#)) - Domine as estruturas de controle

 **DICA:** Pratique no IRB! É a melhor forma de experimentar e aprender novos conceitos.

See also

Guia Rápido Ruby (<https://www.ruby-lang.org/pt/documentation/quickstart>)
Documentação Oficial Ruby (<https://ruby-doc.org>)

Sintaxe Básica do Ruby

Estrutura de um Programa Ruby

Comentários

```
# Comentário de uma linha

=begin
Este é um comentário
de múltiplas linhas
=end
```

Declarações e Expressões

```
# Cada linha geralmente representa uma expressão
nome = "Ruby"
idade = 30

# Ponto e vírgula é opcional (não recomendado)
puts "Olá"; puts "Mundo"

# Quebras de linha são significativas
resultado = 1 +
           2 +
           3    # => 6
```

Convenções de Nomenclatura

Variáveis

```
# Variáveis locais
nome = "João"
primeiro_nome = "Maria"

# Variáveis de instância
```

```
@nome = "João"

# Variáveis de classe
@@contador = 0

# Variáveis globais (evite usar)
$configuracao = "desenvolvimento"

# Constantes (primeira letra maiúscula)
DIAS_SEMANA = 7
PI = 3.14159
```

Identificadores

```
# Classes - CamelCase
class MinhaClasse
end

# Módulos - CamelCase
module MeuModulo
end

# Métodos - snake_case
def calcular_total
end

# Predicados (métodos que retornam boolean)
def maior_de_idade?
end

# Métodos que modificam o objeto (bang methods)
def ordenar!
end
```

Palavras-chave e Estruturas Básicas

Estruturas de Controle

```
# if/else
if idade >= 18
  puts "Maior de idade"
else
  puts "Menor de idade"
end

# unless (if negado)
unless idade < 18
  puts "Maior de idade"
end

# case/when
case idade
when 0..12
  puts "Criança"
when 13..17
  puts "Adolescente"
else
  puts "Adulto"
end
```

Loops

```
# while
contador = 0
while contador < 5
  puts contador
  contador += 1
end

# until (while negado)
contador = 0
until contador >= 5
  puts contador
end
```

```

    contador += 1
end

# for
for i in 0..4
  puts i
end

# each (mais idiomático)
(0..4).each do |i|
  puts i
end

```

Blocos, Procs e Lambdas

Blocos

```

# Bloco com chaves (uma linha)
[1, 2, 3].each { |num| puts num }

# Bloco com do/end (múltiplas linhas)
[1, 2, 3].each do |num|
  dobro = num * 2
  puts dobro
end

```

Procs e Lambdas

```

# Proc
dobrar = Proc.new { |x| x * 2 }
puts dobrar.call(5) # => 10

# Lambda
triplicar = ->(x) { x * 3 }
puts triplicar.call(5) # => 15

```

Operadores

Operadores Aritméticos

```
# Básicos
soma = 5 + 3      # => 8
subtracao = 5 - 3 # => 2
produto = 5 * 3   # => 15
divisao = 5 / 3   # => 1 (divisão inteira)
divisao_float = 5.0 / 3 # => 1.6666...
modulo = 5 % 3    # => 2
exponencial = 2 ** 3 # => 8
```

Operadores de Comparação

```
# Comparações retornam true ou false
5 == 5      # => true
5 != 3      # => true
5 > 3       # => true
5 >= 5      # => true
5 < 8       # => true
5 <= 5      # => true

# Comparação combinada (spaceship operator)
5 <=> 3      # => 1 (maior)
5 <=> 5      # => 0 (igual)
3 <=> 5      # => -1 (menor)
```

Operadores Lógicos

```
# AND
true && true   # => true
true and true # => true (baixa precedência)

# OR
false || true  # => true
false or true  # => true (baixa precedência)
```

```
# NOT
!true          # => false
not true       # => false (baixa precedência)
```

Boas Práticas

1. Use 2 espaços para indentação (não use tabs)
2. Evite ponto e vírgula no final das linhas
3. Use snake_case para métodos e variáveis
4. Use CamelCase para classes e módulos
5. Prefira {} para blocos de uma linha e do/end para múltiplas linhas
6. Evite variáveis globais
7. Nomeie predicados com ? no final
8. Nomeie métodos que modificam o objeto com ! no final

Exemplos Práticos

Calculadora Simples

```
def calculadora(a, b, operacao)
  case operacao
  when '+'
    a + b
  when '-'
    a - b
  when '*'
    a * b
  when '/'
    b.zero? ? "Divisão por zero!" : a.to_f / b
  else
```

```

    "Operação inválida"
  end
end

# Uso
puts calculadora(10, 5, '+') # => 15
puts calculadora(10, 5, '/') # => 2.0

```

Manipulação de Strings

```

def formatar_nome(nome)
  return "Nome inválido" if nome.nil? || nome.empty?

  nome
    .strip
    .split
    .map(&:capitalize)
    .join(' ')
end

# Uso
puts formatar_nome("joão da silva") # => "João Da Silva"
puts formatar_nome("")              # => "Nome inválido"

```

See also

Documentação Ruby (<https://www.ruby-lang.org/pt/documentation/>)
 Guia de Estilo Ruby (<https://ruby-style-guide.shopify.dev/>)

Tipos de Dados em Ruby

Ruby é uma linguagem dinamicamente tipada, o que significa que você não precisa declarar explicitamente o tipo de uma variável. O tipo é determinado pelo valor atribuído.

Tipos Básicos

Numbers (Números)

```
# Integers (Números Inteiros)
idade = 25
ano = 2024
numero_negativo = -42

# Floats (Números Decimais)
altura = 1.75
pi = 3.14159
temperatura = -10.5

# Operações com números
soma = 10 + 5          # => 15
divisao = 10.0 / 3     # => 3.3333...
potencia = 2 ** 3      # => 8
```

Strings (Texto)

```
# Strings com aspas simples
nome = 'Ruby'

# Strings com aspas duplas (permite interpolação)
linguagem = "#{nome} é incrível!"

# Strings multilinhas
descricao = <<-TEXT
  Esta é uma string
  com múltiplas linhas
  em Ruby
```

TEXT

```
# Métodos comuns de strings
nome.upcase      # => "RUBY"
nome.downcase    # => "ruby"
nome.length      # => 4
nome.reverse     # => "ybuR"
```

Symbols (Símbolos)

```
# Símbolos são strings imutáveis, frequentemente usados como
identificadores
:status
:nome
:erro_404

# Comparação de performance
"string" == "string" # Compara dois objetos diferentes
:symbol == :symbol   # Compara o mesmo objeto
```

Booleans (Booleanos)

```
# Apenas true e false
verdadeiro = true
falso = false

# Operadores booleanos
true && false # => false
true || false # => true
!true         # => false

# Valores "truthy" e "falsy"
# Em Ruby, apenas false e nil são considerados falsy
# Todo o resto é truthy
```

Nil (Nulo)

```
# nil representa a ausência de valor
variavel = nil

# Verificando nil
variavel.nil?      # => true
variavel == nil    # => true
```

Tipos de Coleção

Arrays (Vetores)

```
# Criando arrays
numeros = [1, 2, 3, 4, 5]
misturado = [1, "dois", :tres, 4.0]

# Acessando elementos
primeiro = numeros[0]    # => 1
ultimo = numeros[-1]     # => 5

# Modificando arrays
numeros << 6              # Adiciona ao final
numeros.push(7)           # Também adiciona ao final
numeros.pop               # Remove do final
numeros.shift             # Remove do início
numeros.unshift(0)        # Adiciona no início
```

Hashes (Dicionários)

```
# Criando hashes
pessoa = {
  nome: "João",
  idade: 30,
  cidade: "São Paulo"
}

# Sintaxe antiga (ainda válida)
```

```

pessoa_antiga = {
  "nome" => "João",
  "idade" => 30
}

# Acessando valores
nome = pessoa[:nome]      # => "João"
idade = pessoa[:idade]    # => 30

# Modificando hashes
pessoa[:profissao] = "Desenvolvedor"
pessoa.delete(:cidade)

```

Ranges (Intervalos)

```

# Intervalos inclusivos
numeros = 1..5           # Inclui 5 (1, 2, 3, 4, 5)

# Intervalos exclusivos
letras = 'a'...'d'       # Não inclui 'd' (a, b, c)

# Usando ranges
numeros.to_a             # => [1, 2, 3, 4, 5]
letras.include?('c')     # => true

```

Conversão entre Tipos

```

# String para Número
"123".to_i               # => 123 (inteiro)
"3.14".to_f              # => 3.14 (float)

# Número para String
123.to_s                 # => "123"
3.14.to_s                 # => "3.14"

# Array para String

```

```
[1, 2, 3].join(", ") # => "1, 2, 3"

# String para Array
"a,b,c".split(",") # => ["a", "b", "c"]
```

Verificação de Tipos

```
# Verificando o tipo de um objeto
1.class # => Integer
"texto".class # => String
[1, 2, 3].class # => Array

# Verificando se um objeto é de determinado tipo
1.is_a?(Integer) # => true
"texto".is_a?(String) # => true
```

Exemplos Práticos

Manipulação de Diferentes Tipos

```
def processar_dado(dado)
  case dado
  when String
    "Texto: #{dado.upcase}"
  when Integer
    "Número: #{dado * 2}"
  when Array
    "Array com #{dado.length} elementos"
  else
    "Tipo não suportado: #{dado.class}"
  end
end

puts processar_dado("ruby") # => "Texto: RUBY"
```

```
puts processar_dado(42)          # => "Número: 84"  
puts processar_dado([1, 2, 3])  # => "Array com 3 elementos"
```

See also

Documentação Hash (<https://ruby-doc.org/core/Hash.html>)

Documentação String (<https://ruby-doc.org/core/String.html>)

Documentação Array (<https://ruby-doc.org/core/Array.html>)

Strings em Ruby

As strings em Ruby são objetos que representam sequências de caracteres. São extremamente versáteis e oferecem uma grande variedade de métodos úteis.

Criação de Strings

Formas de Declaração

```
# Aspas simples
nome = 'Ruby'

# Aspas duplas (permite interpolação)
versao = "3.2.0"
descricao = "#{nome} versão #{versao}"

# Heredoc (para strings multilinhas)
texto_longo = <<~TEXT
  Esta é uma string multilinha.
  Mantém a formatação
  e permite #{interpolacao}.
TEXT

# Strings com caracteres especiais
caminho = %q{C:\Users\Nome} # Não interpreta caracteres especiais
comando = %Q{ruby -v}      # Interpreta caracteres especiais
```

Interpolação

```
nome = "Alice"
idade = 25

# Interpolação básica
mensagem = "#{nome} tem #{idade} anos"

# Interpolação com expressões
```

```
dobro_idade = "#{nome} terá #{idade * 2} anos em 25 anos"

# Interpolação com métodos
grito = "#{nome.upcase} ESTÁ GRITANDO!"
```

Métodos Comuns

Transformação

```
texto = "ruby é incrível"

texto.upcase      # => "RUBY É INCRÍVEL"
texto.downcase    # => "ruby é incrível"
texto.capitalize  # => "Ruby é incrível"
texto.swapcase    # => "RUBY É INCRÍVEL"
texto.reverse     # => "levírcni é ybur"
```

Busca e Substituição

```
frase = "O Ruby é uma linguagem Ruby"

# Substituição
frase.sub('Ruby', 'Python')      # => "O Python é uma linguagem
Ruby"
frase.gsub('Ruby', 'Python')     # => "O Python é uma linguagem
Python"

# Busca
frase.include?('Ruby')           # => true
frase.start_with?('O')          # => true
frase.end_with?('Ruby')          # => true
frase.index('Ruby')              # => 2
```

Manipulação


```

# Split e Join
palavras = "ruby,python,javascript".split(',') # => ["ruby",
"python", "javascript"]
palavras.join(' e ') # => "ruby e python
e javascript"

# Strip (remove espaços em branco)
" ruby ".strip # => "ruby"
" ruby ".lstrip # => "ruby "
" ruby ".rstrip # => " ruby"

# Padding
"ruby".ljust(10) # => "ruby      "
"ruby".rjust(10) # => "      ruby"
"ruby".center(10) # => "  ruby  "

```

Comparação de Strings

```

# Comparação básica
"ruby" == "ruby" # => true
"ruby" === "ruby" # => true
"ruby" <=> "python" # => 1 (comparação lexicográfica)

# Ignorando case
"Ruby".casecmp("ruby") # => true
"Ruby".downcase == "ruby" # => true

```

Codificação e Encoding

```

# Encoding
texto = "こんにちは"
texto.encoding # => #<Encoding:UTF-8>
texto.encode('EUC-JP') # Converte para EUC-JP

```

```
# Força encoding
texto.force_encoding('UTF-8')
```

Exemplos Práticos

Manipulação de Texto

```
def formatar_nome(nome)
  # Remove espaços extras e capitaliza cada palavra
  nome.strip.split.map(&:capitalize).join(' ')
end

puts formatar_nome(" ada lovelace ") # => "Ada Lovelace"

def extrair_dominio(email)
  email.split('@').last
end

puts extrair_dominio("usuario@exemplo.com") # => "exemplo.com"
```

Validações Simples

```
def email_valido?(email)
  email.include?('@') &&
  email.include?('.') &&
  !email.start_with?('@') &&
  !email.end_with?('.')
end

puts email_valido?("user@domain.com") # => true
puts email_valido?("invalid.email") # => false
```

Formatação de Texto

```
def formatar_cpf(cpf)
  numeros = cpf.gsub(/\D/, '') # Remove não-dígitos
```

```
    "#{numeros[0..2]}.#{numeros[3..5]}.#{numeros[6..8]}-#{  
    {numeros[9..10]}"  
end  
  
puts formatar_cpf("12345678901") # => "123.456.789-01"
```

Performance e Boas Práticas

```
# Uso eficiente de strings  
# Ruim  
string = ""  
10.times { string = string + "a" }  
  
# Bom  
string = ""  
10.times { string << "a" } # Mais eficiente  
  
# Freeze para strings imutáveis  
CONSTANT_STRING = "não muda".freeze
```

See also

Ruby 3.2 String API (<https://rubyapi.org/3.2/o/string>)

Documentação Oficial de String (<https://ruby-doc.org/core/String.html>)

Tipos Numéricos em Ruby

Ruby oferece diversos tipos numéricos para diferentes necessidades, desde cálculos simples até operações matemáticas complexas.

Tipos Básicos

Integers (Números Inteiros)

```
# Integers podem ser de qualquer tamanho
pequeno = 42
grande = 12345678901234567890

# Diferentes bases
binario = 0b1010      # => 10
octal = 0o252          # => 170
hexadecimal = 0xAB     # => 171

# Underscore para legibilidade
milhao = 1_000_000     # => 1000000
```

Floats (Números de Ponto Flutuante)

```
# Declaração básica
pi = 3.14159
cientifico = 1.2e-3    # => 0.0012

# Precisão
resultado = 0.1 + 0.2  # => 0.30000000000000004
```

Rational (Números Racionais)

```
# Criação de números racionais
racional = Rational(3, 4)  # => (3/4)
r = 3/4r                  # Sintaxe curta
```

```
# Operações com racionais
soma = Rational(1, 2) + Rational(1, 3) # => (5/6)
```

Complex (Números Complexos)

```
# Criação de números complexos
complexo = Complex(2, 3) # => (2+3i)
c = 2 + 3i # Sintaxe curta

# Operações com complexos
modulo = complexo.abs # => 3.605551275463989
```

Operações Matemáticas

Operações Básicas

```
# Aritméticas fundamentais
soma = 5 + 3 # => 8
subtracao = 5 - 3 # => 2
multiplicacao = 5 * 3 # => 15
divisao = 5 / 3 # => 1 (divisão inteira)
divisao_float = 5.0 / 3 # => 1.6666666666666667
modulo = 5 % 3 # => 2
exponencial = 2 ** 3 # => 8
```

Métodos Matemáticos

```
# Métodos da classe Math
raiz = Math.sqrt(16) # => 4.0
seno = Math.sin(Math::PI/2) # => 1.0
log = Math.log(100, 10) # => 2.0
```

Conversões

```

# Entre tipos numéricos
inteiro = 42
float = inteiro.to_f      # => 42.0
racional = inteiro.to_r   # => (42/1)
complexo = inteiro.to_c   # => (42+0i)

# De string para número
"123".to_i      # => 123
"3.14".to_f     # => 3.14
"0xFF".to_i(16) # => 255

```

Comparações e Verificações

```

# Operadores de comparação
5 < 10      # => true
5 >= 5      # => true
3.14 == 3    # => false

# Verificações de tipo
42.integer? # => true
3.14.float? # => true
(2/3r).rational? # => true

```

Arredondamento e Precisão

```

# Métodos de arredondamento
3.14159.round(2)  # => 3.14
3.14159.ceil      # => 4
3.14159.floor     # => 3
3.14159.truncate  # => 3

# Precisão decimal
require 'bigdecimal'
BigDecimal('3.14159').round(2) # => 3.14

```

Exemplos Práticos

Cálculos Financeiros

```
def calcular_juros(principal, taxa, tempo)
  principal * (1 + taxa) ** tempo
end

investimento = calcular_juros(1000, 0.05, 3)
puts format("%.2f", investimento) # => 1157.63
```

Operações Matemáticas Complexas

```
def distancia_entre_pontos(x1, y1, x2, y2)
  Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
end

dist = distancia_entre_pontos(0, 0, 3, 4)
puts dist # => 5.0
```

Manipulação de Precisão

```
def media_com_precisao(numeros)
  soma = numeros.reduce(0, :+)
  (soma.to_f / numeros.length).round(2)
end

notas = [7.5, 8.2, 6.9, 9.3]
puts media_com_precisao(notas) # => 7.98
```

Boas Práticas

```
# Use integers para contadores
contador = 0
contador += 1
```

```
# Use floats para cálculos científicos
velocidade = 299_792_458.0 # m/s

# Use rational para frações exatas
proporcao = Rational(1, 3)

# Use BigDecimal para cálculos financeiros
require 'bigdecimal'
valor = BigDecimal('10.99')
```

See also

Documentação de Math (<https://ruby-doc.org/core/Math.html>)

Documentação de Float (<https://ruby-doc.org/core/Float.html>)

Documentação de Integer (<https://ruby-doc.org/core/Integer.html>)

Arrays e Hashes em Ruby

Arrays e Hashes são estruturas de dados fundamentais em Ruby, permitindo armazenar e organizar coleções de objetos.

Arrays

Arrays são coleções ordenadas de objetos que podem ser acessados por índice.

Criação de Arrays

```
# Formas de criar arrays
array_vazio = []
numeros = [1, 2, 3, 4, 5]
misturado = [1, "dois", 3.0, true]
array_palavras = %w[ruby python javascript] # => ["ruby",
"python", "javascript"]
array_simbolos = %i[nome idade cidade]      # => [:nome, :idade,
:cidade]

# Array com construtor
Array.new(3) # => [nil, nil, nil]
Array.new(3, "ruby") # => ["ruby", "ruby", "ruby"]
```

Acessando Elementos

```
numeros = [1, 2, 3, 4, 5]

# Índices positivos (do início)
numeros[0] # => 1
numeros.first # => 1
numeros[2] # => 3

# Índices negativos (do fim)
numeros[-1] # => 5
numeros.last # => 5
numeros[-2] # => 4
```

```
# Ranges
numeros[1..3]    # => [2, 3, 4]
numeros[1...3]   # => [2, 3]
```

Modificando Arrays

```
lista = [1, 2, 3]

# Adicionando elementos
lista << 4          # => [1, 2, 3, 4]
lista.push(5)       # => [1, 2, 3, 4, 5]
lista.unshift(0)    # => [0, 1, 2, 3, 4, 5]

# Removendo elementos
lista.pop           # => 5
lista.shift         # => 0
lista.delete(2)     # Remove o elemento 2

# Modificando elementos
lista[0] = "primeiro"
```

Métodos Úteis para Arrays

```
numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

# Ordenação
numeros.sort        # => [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
numeros.sort.uniq   # => [1, 2, 3, 4, 5, 6, 9]

# Transformação
numeros.map { |n| n * 2 } # => [6, 2, 8, 2, 10, 18, 4, 12, 10, 6]
numeros.select { |n| n > 4 } # => [5, 9, 6, 5]

# Iteração
```

```
numeros.each { |n| puts n }  
numeros.each_with_index { |n, i| puts "#{i}: #{n}" }
```

Hashes

Hashes são coleções de pares chave-valor, similares a dicionários em outras linguagens.

Criação de Hashes

```
# Diferentes sintaxes  
hash1 = {}  
hash2 = Hash.new  
hash3 = { "nome" => "Ruby", "versao" => 3.2 }  
hash4 = { nome: "Ruby", versao: 3.2 } # Sintaxe moderna com  
símbolos  
  
# Hash com valor padrão  
scores = Hash.new(0) # Valor padrão para chaves inexistentes
```

Acessando e Modificando

```
pessoa = { nome: "Alice", idade: 30, cidade: "São Paulo" }  
  
# Acessando valores  
pessoa[:nome] # => "Alice"  
pessoa.fetch(:idade) # => 30  
  
# Modificando valores  
pessoa[:idade] = 31  
pessoa[:profissao] = "Desenvolvedora"  
  
# Removendo pares  
pessoa.delete(:cidade)
```

Métodos Úteis para Hashes

```

# Iteração
pessoa.each do |chave, valor|
  puts "#{chave}: #{valor}"
end

# Chaves e valores
pessoa.keys      # => [:nome, :idade, :profissao]
pessoa.values    # => ["Alice", 31, "Desenvolvedora"]

# Transformação
pessoa.transform_values { |v| v.to_s }
pessoa.select { |k, v| v.is_a?(String) }

```

Combinando Arrays e Hashes

Arrays de Hashes

```

usuarios = [
  { id: 1, nome: "Alice", admin: true },
  { id: 2, nome: "Bob", admin: false },
  { id: 3, nome: "Carol", admin: true }
]

# Encontrando usuários
admins = usuarios.select { |u| u[:admin] }
nomes = usuarios.map { |u| u[:nome] }

```

Hash com Arrays

```

biblioteca = {
  ficção: ["1984", "Fundação", "Neuromancer"],
  técnico: ["Clean Code", "Ruby Design Patterns"],
  poesia: ["O Corvo", "Os Lusíadas"]
}

# Manipulando a biblioteca

```

```
biblioteca[:ficção] << "Duna"
todos_livros = biblioteca.values.flatten
```

Boas Práticas

```
# Use símbolos como chaves de hash
config = {
  porta: 3000,
  host: "localhost",
  debug: true
}

# Use map ao invés de each para transformações
nomes = ["alice", "bob", "carol"]
nomes_maiusculos = nomes.map(&:upcase)

# Use select/reject para filtrar
numeros = [1, 2, 3, 4, 5, 6]
pares = numeros.select(&:even?)
impares = numeros.reject(&:even?)
```

See also

Documentação de Array (<https://ruby-doc.org/core/Array.html>)

Documentação de Hash (<https://ruby-doc.org/core/Hash.html>)

Ranges em Ruby

Ranges representam intervalos de valores em Ruby. São úteis para expressar sequências e intervalos de forma concisa.

Tipos de Ranges

Range Inclusivo (..)

```
# Inclui o último número
intervalo = 1..5 # inclui 1, 2, 3, 4, 5
letras = 'a'..'e' # inclui 'a', 'b', 'c', 'd', 'e'

# Verificando elementos
puts intervalo.include?(3) # => true
puts intervalo.include?(6) # => false
```

Range Exclusivo (...)

```
# Exclui o último número
intervalo = 1...5 # inclui 1, 2, 3, 4
letras = 'a'...'e' # inclui 'a', 'b', 'c', 'd'

# Verificando elementos
puts intervalo.include?(4) # => true
puts intervalo.include?(5) # => false
```

Operações com Ranges

Conversão para Array

```
# Convertendo ranges em arrays
numeros = (1..5).to_a # => [1, 2, 3, 4, 5]
alfabeto = ('a'..'e').to_a # => ['a', 'b', 'c', 'd', 'e']
```

```
# Ranges com steps
pares = (0..10).step(2).to_a # => [0, 2, 4, 6, 8, 10]
```

Iteração

```
# Iterando sobre ranges
(1..5).each { |n| puts n }

# Com step
(0..10).step(2) { |n| puts "Número par: #{n}" }

# Enumeração
soma = (1..100).reduce(:+) # Soma todos os números de 1 a 100
```

Usos Comuns

Em Condicionais

```
idade = 25

case idade
when 0..12
  puts "Criança"
when 13..17
  puts "Adolescente"
when 18..64
  puts "Adulto"
else
  puts "Idoso"
end
```

Em Arrays

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Slicing com ranges
```

```
primeiros = numeros[0..2]    # => [1, 2, 3]
meio = numeros[3..7]        # => [4, 5, 6, 7]
ultimos = numeros[-3..-1]   # => [8, 9, 10]
```

Em Strings

```
texto = "Ruby é incrível!"

# Substring com ranges
puts texto[0..3]    # => "Ruby"
puts texto[5..6]    # => "é"
puts texto[-9..-1]  # => "incrível!"
```

Métodos Úteis

```
range = (1..10)

# Verificações
range.begin    # => 1
range.end      # => 10
range.exclude_end? # => false (é inclusivo)

# Operações
range.first    # => 1
range.first(3) # => [1, 2, 3]
range.last     # => 10
range.last(3)  # => [8, 9, 10]

# Iteração reversa
range.reverse_each { |n| puts n }
```

Ranges Infinitos

```
# Range sem fim (Ruby 2.6+)
numeros = (1..)
```



```

letras = ('a'..)

# Primeiros elementos
puts numeros.first(5) # => [1, 2, 3, 4, 5]
puts letras.first(3)  # => ["a", "b", "c"]

# Range com início infinito (Ruby 2.7+)
negativos = (..0)
puts negativos.include?(-1) # => true
puts negativos.include?(1)  # => false

```

Boas Práticas

```

# Use ranges para intervalos numéricos
def classificar_temperatura(temp)
  case temp
  when -Float::INFINITY..0
    "Congelando"
  when 0..20
    "Frio"
  when 21..25
    "Agradável"
  when 26..35
    "Quente"
  else
    "Muito quente"
  end
end

# Use ranges para validações
def validar_idade(idade)
  (0..120).include?(idade)
end

# Use ranges para gerar sequências
def gerar_codigo

```

```
( 'A' .. 'Z' ).to_a.sample(6).join  
end
```

See also

Documentação de Range (<https://ruby-doc.org/core/Range.html>)

Entrada e Saída em Ruby

Ruby oferece um conjunto robusto de ferramentas para manipulação de entrada e saída (I/O), permitindo interação com usuários, arquivos e streams de dados.

Visão Geral

A entrada e saída em Ruby é baseada em streams, que são sequências de dados que podem ser lidas ou escritas. Os principais tipos são:

- **Entrada Padrão (STDIN)**: Para ler dados do teclado
- **Saída Padrão (STDOUT)**: Para exibir dados na tela
- **Saída de Erro (STDERR)**: Para mensagens de erro
- **Arquivos**: Para persistência de dados

Tópicos Principais

Entrada e Saída Básica

- Métodos `puts`, `print` e `p`
- Leitura com `gets`
- Formatação de strings
- Manipulação de entrada do usuário

Trabalhando com Arquivos

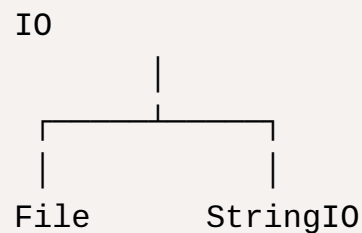
- Leitura e escrita de arquivos
- Manipulação de diretórios
- Streams de dados
- Tratamento de erros

Conceitos Fundamentais

```
# Streams padrão
STDIN  # Entrada padrão
STDOUT # Saída padrão
STDERR # Saída de erro

# Aliases globais
$stdin # Mesmo que STDIN
$stdout # Mesmo que STDOUT
$stderr # Mesmo que STDERR
```

Hierarquia de Classes



Casos de Uso Comuns

```
# Entrada/Saída básica
puts "Digite seu nome:"
nome = gets.chomp
puts "Olá, #{nome}!"

# Manipulação de arquivos
File.open("dados.txt", "w") do |arquivo|
  arquivo.puts "Dados importantes"
end

# Redirecionamento de saída
$stdout = File.new("log.txt", "w")
```

```
puts "Isso vai para o arquivo"  
$stdout = STDOUT # Restaura saída padrão
```

Boas Práticas

- Sempre feche recursos abertos
- Use blocos com `File.open`
- Trate erros apropriadamente
- Valide entrada do usuário
- Forneça feedback claro

Próximos Passos

Explore os subtópicos para aprender mais sobre:

- Entrada e Saída Básica ([Entrada e Saída Básica em Ruby](https://ruby-doc.org/core/File.html))
- Trabalhando com Arquivos ([Trabalhando com Arquivos em Ruby](https://ruby-doc.org/core/IO.html))

See also

Documentação File (<https://ruby-doc.org/core/File.html>)

Documentação IO (<https://ruby-doc.org/core/IO.html>)

Entrada e Saída Básica em Ruby

Saída de Dados

puts vs print vs p

```
# puts - adiciona nova linha automaticamente
puts "Olá, Mundo!" # => Olá, Mundo!
                  # => (nova linha)

# print - não adiciona nova linha
print "Olá, "
print "Mundo!"      # => Olá, Mundo!(sem nova linha)

# p - mostra representação mais detalhada
p "Olá\nMundo"      # => "Olá\nMundo"
p [1, 2, 3]          # => [1, 2, 3]
```

Formatação de Strings

```
nome = "Ruby"
versao = 3.2

# Interpolação
puts "#{nome} versão #{versao}"

# printf
printf("%-10s %04d\n", "Ruby", 42) # => Ruby      0042
printf("Pi: %.2f\n", Math::PI)    # => Pi: 3.14

# format (sprintf)
puts format("%.2f%%", 99.9999)     # => 100.00%
```

Entrada de Dados

gets e STDIN

```
# Leitura básica
print "Digite seu nome: "
nome = gets          # Lê uma linha (inclui \n)
nome = gets.chomp    # Lê uma linha (remove \n)

# Leitura segura do STDIN
input = STDIN.gets

# Leitura com timeout
require 'timeout'
begin
  puts "Você tem 5 segundos..."
  input = Timeout::timeout(5) { gets }
rescue Timeout::Error
  puts "Tempo esgotado!"
end
```

Leitura de Diferentes Tipos

```
# Lendo números
print "Digite um número: "
numero = gets.chomp.to_i    # Para inteiros
numero = gets.chomp.to_f    # Para float

# Lendo múltiplos valores
print "Digite dois números (separados por espaço): "
x, y = gets.chomp.split.map(&:to_i)
```

Entrada/Saída com ARGV

```
# script.rb
if ARGV.empty?
  puts "Uso: ruby script.rb <nome>"
  exit
end
```

```

end

nome = ARGV[0]
puts "Olá, #{nome}!"

# Terminal: ruby script.rb Alice
# => Olá, Alice!

```

Manipulação de Erros

```

# Saída de erro
STDERR.puts "Erro: arquivo não encontrado"

# Redirecionamento de saída
$stdout = File.new('saida.log', 'w')
puts "Isso vai para o arquivo"
$stdout = STDOUT # Restaura saída padrão

```

Interatividade

```

def menu
  loop do
    print "\nEscolha uma opção:\n" \
          "1. Novo\n" \
          "2. Abrir\n" \
          "3. Sair\n" \
          "Opção: "

    case gets.chomp
    when '1' then puts "Criando novo..."
    when '2' then puts "Abrindo..."
    when '3' then break
    else puts "Opção inválida!"
    end
  end
end

```



```

end

# Interface simples
def confirmar(mensagem)
  print "#{mensagem} (s/n): "
  gets.chomp.downcase == 's'
end

```

Boas Práticas

```

# Validação de entrada
def ler_numero
  print "Digite um número: "
  Integer(gets.chomp)
rescue ArgumentError
  puts "Entrada inválida! Tente novamente."
  retry
end

# Mensagens claras
def solicitar_entrada(prompt)
  print "#{prompt}: "
  gets.chomp
end

# Feedback ao usuário
def processar_dados(dados)
  print "Processando..."
  resultado = dados.upcase
  puts "concluído!"
  resultado
end

```

Exemplos Práticos

```

# Calculadora simples
def calculadora
  print "Digite o primeiro número: "
  n1 = gets.chomp.to_f

  print "Digite a operação (+, -, *, /): "
  op = gets.chomp

  print "Digite o segundo número: "
  n2 = gets.chomp.to_f

  resultado = case op
  when '+' then n1 + n2
  when '-' then n1 - n2
  when '*' then n1 * n2
  when '/' then n1 / n2
  else "Operação inválida"
  end

  puts "Resultado: #{resultado}"
end

# Leitor de senhas
def ler_senha
  require 'io/console'
  print "Digite sua senha: "
  STDIN.noecho(&gets).chomp
end

```

See also

Documentação de IO (<https://ruby-doc.org/core/IO.html>)

Documentação de Kernel (<https://ruby-doc.org/core/Kernel.html>)

Trabalhando com Arquivos em Ruby

Abrindo e Fechando Arquivos

Métodos Básicos

```
# Abrindo um arquivo (modo leitura)
arquivo = File.open('exemplo.txt', 'r')
conteudo = arquivo.read
arquivo.close

# Usando bloco (fecha automaticamente)
File.open('exemplo.txt', 'r') do |arquivo|
  conteudo = arquivo.read
end
```

Modos de Abertura

```
# Modos comuns
'r' # Somente leitura
'w' # Escrita (cria novo/sobrescreve)
'a' # Anexar ao final
'r+' # Leitura e escrita
'w+' # Leitura e escrita (cria novo/sobrescreve)
'a+' # Leitura e anexar

# Exemplo com diferentes modos
File.open('log.txt', 'a') do |arquivo|
  arquivo.puts "Nova entrada: #{Time.now}"
end
```

Leitura de Arquivos

Lendo Conteúdo

```
# Lendo arquivo inteiro
conteudo = File.read('arquivo.txt')

# Lendo linhas em array
linhas = File.readlines('arquivo.txt')

# Lendo linha por linha
File.foreach('arquivo.txt') do |linha|
  puts linha
end

# Lendo com encoding específico
conteudo = File.read('arquivo.txt', encoding: 'UTF-8')
```

Técnicas de Leitura

```
File.open('dados.txt', 'r') do |arquivo|
  # Lê primeiros 10 bytes
  dados = arquivo.read(10)

  # Lê próxima linha
  linha = arquivo.gets

  # Lê até encontrar padrão
  texto = arquivo.readline until texto =~ /fim/

  # Posicionamento no arquivo
  arquivo.seek(20, IO::SEEK_SET)
  arquivo.rewind # Volta ao início
end
```

Escrita em Arquivos

Métodos de Escrita

```

# Escrita simples
File.write('saida.txt', 'Olá, Mundo!')

# Anexando conteúdo
File.write('log.txt', 'Nova entrada\n', mode: 'a')

# Usando write e puts
File.open('dados.txt', 'w') do |arquivo|
  arquivo.write("Linha sem quebra")
  arquivo.puts("Linha com quebra")
  arquivo.printf("Formatado: %s\n", "texto")
end

```

Escrita Estruturada

```

# Escrevendo dados estruturados
dados = {nome: 'Ruby', versao: '3.2.0'}

File.open('config.txt', 'w') do |arquivo|
  dados.each do |chave, valor|
    arquivo.puts "#{chave}: #{valor}"
  end
end

```

Manipulação de Arquivos

Operações com Arquivos

```

# Verificações
File.exist?('arquivo.txt')    # Existe?
File.directory?('pasta')     # É diretório?
File.file?('arquivo.txt')    # É arquivo?
File.zero?('vazio.txt')      # Está vazio?

# Informações
File.size('arquivo.txt')     # Tamanho em bytes

```

```
File.mtime('arquivo.txt')      # Data de modificação
File.dirname('path/arquivo.txt') # Diretório
File.basename('path/arquivo.txt') # Nome do arquivo
```

Manipulação de Diretórios

```
# Criando diretórios
Dir.mkdir('nova_pasta')
FileUtils.mkdir_p('path/to/nested/folder')

# Listando arquivos
Dir.entries('.')          # Lista tudo
Dir['*.txt']              # Com glob pattern
Dir.glob('**/*.rb')       # Recursivo

# Navegando em diretórios
Dir.chdir('nova_pasta') do
  # Trabalha no diretório
end
```

Tratamento de Erros

```
begin
  arquivo = File.open('inexistente.txt')
rescue Errno::ENOENT
  puts "Arquivo não encontrado!"
rescue Errno::EACCES
  puts "Permissão negada!"
ensure
  arquivo&.close
end
```

Boas Práticas

```

# Use blocos para garantir fechamento
def processar_arquivo(nome)
  File.open(nome, 'r') do |arquivo|
    yield arquivo
  end
end

# Backup antes de modificar
def atualizar_arquivo(nome)
  FileUtils.cp(nome, "#{nome}.bak")
  File.write(nome, novo_conteudo)
end

# Validação de arquivo
def arquivo_valido?(nome)
  File.file?(nome) && File.readable?(nome)
end

```

Exemplos Práticos

```

# Logger simples
class Logger
  def initialize(arquivo)
    @arquivo = arquivo
  end

  def log(mensagem)
    File.open(@arquivo, 'a') do |f|
      f.puts "#{Time.now} - #{mensagem}"
    end
  end
end

# Processador CSV
def processar_csv(arquivo)
  require 'csv'

```

```
CSV.foreach(arquivo, headers: true) do |linha|
  yield linha
end

# Leitor de configuração
def ler_config(arquivo)
  require 'yaml'
  YAML.load_file(arquivo)
end
```

See also

Documentação de Dir (<https://ruby-doc.org/core/Dir.html>)
Documentação de File (<https://ruby-doc.org/core/File.html>)
Documentação de FileUtils (<https://ruby-doc.org/stdlib/libdoc/fileutils/rdoc/FileUtils.html>)

Estruturas de Controle em Ruby

As estruturas de controle em Ruby permitem controlar o fluxo de execução do programa de forma elegante e expressiva.

Visão Geral

Ruby oferece diversas estruturas de controle que podem ser categorizadas em:

- Estruturas condicionais
- Estruturas de repetição
- Estruturas de salto

Estruturas Condicionais

if/else/elsif

```
if idade >= 18
  puts "Maior de idade"
elsif idade >= 13
  puts "Adolescente"
else
  puts "Criança"
end

# Forma modificadora (one-liner)
puts "Pode dirigir" if idade >= 18
```

unless

```
unless temperatura > 30
  puts "Está agradável"
end
```

```
# Forma modificadora
puts "Precisa de casaco" unless temperatura > 20
```

case/when

```
case idade
when 0..12
  puts "Criança"
when 13..17
  puts "Adolescente"
when 18..64
  puts "Adulto"
else
  puts "Idoso"
end

# Forma concisa com atribuição
status = case idade
  when 0..17 then "Menor"
  when 18..64 then "Adulto"
  else "Idoso"
end
```

Estruturas de Repetição

while

```
contador = 0
while contador < 5
  puts contador
  contador += 1
end

# Forma modificadora
contador += 1 while contador < 5
```

until

```
numero = 1
until numero > 10
  puts numero
  numero *= 2
end

# Forma modificadora
numero *= 2 until numero > 10
```

for

```
for i in 1..5
  puts i
end

for item in ['a', 'b', 'c']
  puts item
end
```

loop

```
loop do
  print "Digite 'sair' para encerrar: "
  comando = gets.chomp
  break if comando == 'sair'
end
```

Estruturas de Salto

break

```
10.times do |i|
  break if i > 5
end
```

```
puts i
end
```

next

```
5.times do |i|
  next if i.even?
  puts "Número ímpar: #{i}"
end
```

redo

```
count = 0
for i in 0..5
  puts i
  if i == 2 && count == 0
    count += 1
    redo # Repete a iteração atual
  end
end
```

retry

```
tentativas = 0
begin
  # Código que pode falhar
  raise "erro" if tentativas == 0
rescue
  tentativas += 1
  retry if tentativas < 3
end
```

Blocos begin/end

begin/rescue/ensure

```

begin
  # Código que pode gerar erro
  arquivo = File.open("nao_existe.txt")
rescue Errno::ENOENT
  puts "Arquivo não encontrado"
rescue => e
  puts "Outro erro: #{e.message}"
ensure
  # Sempre executado
  arquivo&.close
end

```

Boas Práticas

```

# Prefira unless a if !
# Ruim
if !arquivo.existe?
  puts "Arquivo não encontrado"
end

# Melhor
unless arquivo.existe?
  puts "Arquivo não encontrado"
end

# Use modificadores para condições simples
puts "Par" if numero.even?

# Evite else com unless
# Ruim
unless idade >= 18
  puts "Menor"
else
  puts "Adulto"
end

```

```

# Melhor
if idade >= 18
  puts "Adulto"
else
  puts "Menor"
end

# Use case quando houver múltiplas condições
# Ruim
if status == :pendente
  processar_pendente
elsif status == :aprovado
  processar_aprovado
elsif status == :rejeitado
  processar_rejeitado
end

# Melhor
case status
when :pendente then processar_pendente
when :aprovado then processar_aprovado
when :rejeitado then processar_rejeitado
end

```

Exemplos Práticos

```

def processar_pedido(pedido)
  case pedido.status
  when :novo
    validar_pedido(pedido)
  when :validado
    calcular_total(pedido)
  when :pago
    enviar_confirmacao(pedido)
  else
    raise "Status inválido: #{pedido.status}"
  end
end

```

```
end
end

def ler_arquivo_seguro(caminho)
  return unless File.exist?(caminho)

  begin
    File.read(caminho)
  rescue => e
    puts "Erro ao ler arquivo: #{e.message}"
    nil
  end
end
end
```

See also

Documentação de Estruturas de Controle (https://ruby-doc.org/core/doc/syntax/control_expressions_rdoc.html)

Documentação de Exceções (https://ruby-doc.org/core/doc/syntax/exceptions_rdoc.html)

Operadores em Ruby

Ruby oferece uma rica variedade de operadores que permitem realizar operações matemáticas, comparações, atribuições e operações lógicas.

Operadores Aritméticos

```
# Operações básicas
soma = 5 + 3      # => 8
subtracao = 10 - 4 # => 6
produto = 4 * 3    # => 12
divisao = 15 / 2    # => 7 (divisão inteira)
modulo = 15 % 4     # => 3
exponencial = 2 ** 3 # => 8

# Divisão com ponto flutuante
divisao_float = 15.0 / 2 # => 7.5
divisao_float = 15.fdiv(2) # => 7.5
```

Operadores de Atribuição

```
# Atribuição simples
x = 5

# Atribuição com operação
x += 3 # x = x + 3
x -= 2 # x = x - 2
x *= 4 # x = x * 4
x /= 2 # x = x / 2
x **= 2 # x = x ** 2
x %= 3 # x = x % 3

# Atribuição paralela
```



```
a, b = 1, 2
x, y = y, x # Troca de valores
```

Operadores de Comparação

```
# Igualdade e diferença
5 == 5      # => true
5 != 3      # => true
5 <=> 5      # => 0 (spaceship operator)

# Maior e menor
5 > 3       # => true
5 >= 5      # => true
3 < 5       # => true
3 <= 5      # => true

# Comparação de identidade
a.equal?(b)  # Compara se são o mesmo objeto
"ruby" === String # => false
String === "ruby" # => true (case subsumption)
```

Operadores Lógicos

```
# AND
true && true  # => true
true and true # => true (baixa precedência)

# OR
false || true  # => true
false or true  # => true (baixa precedência)

# NOT
!true          # => false
not true       # => false (baixa precedência)
```

```
# Combinações
(1 < 2) && (2 < 3) # => true
```

Operadores de Range

```
# Range inclusivo
1..5 # inclui 1, 2, 3, 4, 5

# Range exclusivo
1...5 # inclui 1, 2, 3, 4

# Uso com strings
'a'..'d' # => 'a', 'b', 'c', 'd'
```

Operadores Bit a Bit

```
# AND bit a bit
5 & 3 # => 1

# OR bit a bit
5 | 3 # => 7

# XOR bit a bit
5 ^ 3 # => 6

# Deslocamento à esquerda
5 << 1 # => 10

# Deslocamento à direita
5 >> 1 # => 2
```

Operadores Especiais

```

# Operador ternário
idade >= 18 ? "Maior" : "Menor"

# Operador de segurança nula
usuario&.nome # Retorna nil se usuario for nil

# Operador de coalescência nula
nome = nil
nome ||= "Anônimo" # Atribui "Anônimo" se nome for nil

```

Boas Práticas

```

# Use && e || em vez de and e or para lógica
if usuario && usuario.ativo?
  # código aqui
end

# Prefira operador ternário para condições simples
status = idade >= 18 ? "Maior" : "Menor"

# Use operador de segurança nula para evitar NoMethodError
usuario&.endereco&.cidade

# Evite comparações encadeadas sem parênteses
# Ruim
if a < b && b < c
  # código
end

# Melhor
if (a < b) && (b < c)
  # código
end

```

Exemplos Práticos

```

def calcular_desconto(valor, tipo_cliente)
  desconto = case tipo_cliente
    when :vip      then 0.2
    when :regular then 0.1
    else 0.05
  end

  valor * (1 - desconto)
end

def validar_idade(idade)
  return "Idade inválida" unless idade&.positive?
  idade >= 18 ? "Maior de idade" : "Menor de idade"
end

def combinar_strings(str1, str2)
  resultado = str1&.strip
  resultado = "#{resultado} #{str2}".strip if str2
  resultado || "Vazio"
end

```

See also

Precedência de Operadores (https://ruby-doc.org/core/doc/syntax/precedence_rdoc.html)

Documentação de Operadores (https://ruby-doc.org/core/doc/syntax/operators_rdoc.html)

Condicionais em Ruby

As estruturas condicionais em Ruby permitem que seu código tome decisões e execute diferentes blocos de código baseados em condições específicas.

if/else/elsif

Sintaxe Básica

```
if condicao
  # código executado se condição for verdadeira
elsif outra_condicao
  # código executado se outra_condição for verdadeira
else
  # código executado se nenhuma condição for verdadeira
end
```

Exemplos Práticos

```
def verificar_temperatura(temp)
  if temp >= 38
    "Febre alta"
  elsif temp >= 37
    "Febre baixa"
  else
    "Temperatura normal"
  end
end

# Forma modificadora (one-line)
puts "Está quente!" if temperatura > 30
```

unless

Sintaxe Básica

```
unless condicao
  # código executado se condição for falsa
else
  # código executado se condição for verdadeira
end
```

Exemplos Práticos

```
def verificar_acesso(usuario)
  unless usuario.autenticado?
    "Acesso negado"
  else
    "Bem-vindo!"
  end
end

# Forma modificadora
exit unless File.exist?("config.yml")
```

case/when

Sintaxe Básica

```
case expressao
when valor1
  # código para valor1
when valor2, valor3
  # código para valor2 ou valor3
else
  # código padrão
end
```

Exemplos Práticos

```
def classificar_nota(nota)
  case nota
```

```

when 90..100
  "A"
when 80..90
  "B"
when 70..80
  "C"
when 60..70
  "D"
else
  "F"
end
end

# Case com condições
def classificar_idade(idade)
  case
  when idade < 13
    "Criança"
  when idade < 20
    "Adolescente"
  when idade < 65
    "Adulto"
  else
    "Idoso"
  end
end
end

```

Operador Ternário

Sintaxe Básica

```
condicao ? valor_se_verdadeiro : valor_se_falso
```

Exemplos Práticos

```
def status_conta(saldo)
  saldo >= 0 ? "Positivo" : "Negativo"
end

mensagem = idade >= 18 ? "Pode dirigir" : "Não pode dirigir"
```

Boas Práticas

```
# Evite aninhar muitos if/else
# Ruim
def verificar_usuario(usuario)
  if usuario
    if usuario.ativo?
      if usuario.admin?
        "Administrador"
      else
        "Usuário comum"
      end
    else
      "Usuário inativo"
    end
  else
    "Usuário não encontrado"
  end
end

# Melhor
def verificar_usuario(usuario)
  return "Usuário não encontrado" unless usuario
  return "Usuário inativo" unless usuario.ativo?

  usuario.admin? ? "Administrador" : "Usuário comum"
end

# Use case quando houver múltiplas condições
# Ruim
```



```

def traduzir_mes(mes)
  if mes == 1
    "Janeiro"
  elsif mes == 2
    "Fevereiro"
  elsif mes == 3
    "Março"
  # ...
end
end

# Melhor
def traduzir_mes(mes)
  case mes
  when 1 then "Janeiro"
  when 2 then "Fevereiro"
  when 3 then "Março"
  # ...
end
end

```

Exemplos Avançados

```

# Combinando condicionais
def validar_pedido(pedido)
  return false unless pedido && pedido.items.any?

  case
  when pedido.total > 1000 && pedido.cliente.vip?
    aplicar_desconto_vip(pedido)
  when pedido.items.size > 5 || pedido.total > 500
    aplicar_desconto_padrao(pedido)
  else
    pedido
  end
end
end

```

```
# Usando pattern matching (Ruby 2.7+)
def processar_resposta(resposta)
  case resposta
  in { status: 200, data: { nome: String => nome } }
    "Sucesso: #{nome}"
  in { status: 404 }
    "Não encontrado"
  in { status: Integer => code }
    "Erro: #{code}"
  else
    "Resposta inválida"
  end
end
```

Tratamento de Nil

```
# Usando &. (safe navigation operator)
def nome_usuario(usuario)
  usuario&.perfil&.nome || "Anônimo"
end

# Usando fetch com valor padrão
def configuracao(chave)
  config.fetch(chave) { valor_padrao(chave) }
end
```

See also

Documentação de Pattern Matching (https://ruby-doc.org/core/doc/syntax/pattern_matching_rdoc.html)
 Documentação de Expressões de Controle (https://ruby-doc.org/core/doc/syntax/control_expressions_rdoc.html)

Loops em Ruby

Ruby oferece várias formas de criar loops e iterações, cada uma com seus casos de uso específicos.

while e until

while

```
# Estrutura básica
while condicao
  # código a ser executado
end

# Exemplo prático
contador = 0
while contador < 5
  puts contador
  contador += 1
end

# Forma modificadora
puts "Ainda processando..." while processo_ativo?
```

until

```
# Estrutura básica
until condicao
  # código a ser executado
end

# Exemplo prático
tempo = 10
until tempo.zero?
  puts "#{tempo} segundos restantes"
  tempo -= 1
end
```

```
sleep 1
end
```

for e each

for

```
# Iterando sobre range
for i in 0..5
  puts "Número: #{i}"
end

# Iterando sobre array
frutas = ["maçã", "banana", "laranja"]
for fruta in frutas
  puts "Fruta: #{fruta}"
end
```

each

```
# Com arrays
[1, 2, 3].each do |numero|
  puts "O dobro de #{numero} é #{numero * 2}"
end

# Com hashes
usuario = { nome: "João", idade: 30 }
usuario.each do |chave, valor|
  puts "#{chave}: #{valor}"
end

# Forma compacta
(1..5).each { |n| print "#{n} " }
```

loop

```

# Loop infinito com break
loop do
  print "Digite 'sair' para encerrar: "
  entrada = gets.chomp
  break if entrada == 'sair'
end

# Com contador
contador = 0
loop do
  puts contador
  contador += 1
  break if contador >= 5
end

```

Controle de Loop

break, next e redo

```

# break - sai do loop
numeros = [1, 2, 3, 4, 5]
numeros.each do |n|
  break if n > 3
  puts n
end

# next - pula para próxima iteração
numeros.each do |n|
  next if n.even?
  puts n
end

# redo - repete a iteração atual
tentativas = 0
numeros.each do |n|
  tentativas += 1

```

```
redo if falhou?(n) && tentativas < 3  
end
```

Iteradores Especiais

times

```
5.times do |i|  
  puts "Iteração #{i}"  
end  
  
# Forma compacta  
3.times { puts "Olá!" }
```

upto e downto

```
# Contagem crescente  
1.upto(5) do |n|  
  puts n  
end  
  
# Contagem regressiva  
5.downto(1) do |n|  
  puts n  
end
```

step

```
# Contando de 2 em 2  
0.step(10, 2) do |n|  
  puts n  
end  
  
# Números decimais  
0.0.step(1.0, 0.2) { |n| puts n }
```

Exemplos Práticos

Processamento de Dados

```
def processar_lista(items)
  items.each_with_index do |item, index|
    next if item.nil?
    break if index > 100 # limite de segurança

    resultado = processar_item(item)
    redo if resultado == :retry
  end
end

def ler_arquivo(caminho)
  File.foreach(caminho) do |linha|
    next if linha.strip.empty?
    yield linha if block_given?
  end
end
```

Menus Interativos

```
def menu_principal
  loop do
    puts "\n=== Menu Principal ==="
    puts "1. Novo jogo"
    puts "2. Carregar jogo"
    puts "3. Configurações"
    puts "4. Sair"

    print "\nEscolha uma opção: "
    opcao = gets.chomp

    case opcao
    when "1" then novo_jogo
    when "2" then carregar_jogo
```

```
when "3" then configuracoes
when "4" then break
else puts "Opção inválida!"
end
end
end
```

Boas Práticas

```
# Prefira each sobre for
# Não recomendado
for i in array
  # código
end

# Recomendado
array.each do |item|
  # código
end

# Use iteradores específicos quando possível
# Não recomendado
i = 0
while i < 5
  puts i
  i += 1
end

# Recomendado
5.times { |i| puts i }

# Evite loops infinitos acidentais
# Sempre tenha uma condição de saída clara
loop do
  # código
```



```
break if condicao_saida  
end
```

See also

Documentação de Integer (para times, upto, etc) (<https://ruby-doc.org/core/Integer.html>)

Documentação do Enumerable (<https://ruby-doc.org/core/Enumerable.html>)

Blocos e Iteradores em Ruby

Blocos e iteradores são conceitos fundamentais em Ruby que permitem escrever código mais elegante e reutilizável.

Blocos

Sintaxe Básica

```
# Sintaxe com chaves (uma linha)
[1, 2, 3].each { |numero| puts numero }

# Sintaxe do/end (múltiplas linhas)
[1, 2, 3].each do |numero|
  dobro = numero * 2
  puts "O dobro de #{numero} é #{dobro}"
end
```

Yield

```
# Método que aceita um bloco
def executar_tres_vezes
  yield
  yield
  yield
end

executar_tres_vezes { puts "Ruby!" }

# Yield com parâmetros
def processar_item(item)
  puts "Iniciando processamento..."
  yield(item) if block_given?
  puts "Processamento finalizado."
end
```

```
processar_item("dados") { |x| puts "Processando #{x}..." }
```

Blocos como Objetos

```
# Convertendo bloco em Proc
multiplicador = Proc.new { |x| x * 2 }
puts multiplicador.call(5) # => 10

# Lambda
dobrar = ->(x) { x * 2 }
puts dobrar.call(5) # => 10

# Diferenças entre Proc e Lambda
def retorno_proc
  proc = Proc.new { return "Dentro do Proc" }
  proc.call
  "Depois do Proc"
end

def retorno_lambda
  lambda = -> { return "Dentro do Lambda" }
  lambda.call
  "Depois do Lambda"
end
```

Iteradores

Iteradores Básicos

```
# each
[1, 2, 3].each { |n| puts n }

# map/collect
numeros = [1, 2, 3]
dobrados = numeros.map { |n| n * 2 } # => [2, 4, 6]
```

```
# select/find_all
pares = (1..10).select { |n| n.even? } # => [2, 4, 6, 8, 10]

# reject
impares = (1..10).reject { |n| n.even? } # => [1, 3, 5, 7, 9]
```

Iteradores Avançados

```
# reduce/inject
soma = [1, 2, 3, 4].reduce(0) { |acc, n| acc + n } # => 10

# each_with_index
["a", "b", "c"].each_with_index do |letra, index|
  puts "#{index}: #{letra}"
end

# each_with_object
hash = ["a", "b", "c"].each_with_object({}) do |item, hash|
  hash[item] = item.upcase
end
```

Implementando Iteradores Personalizados

```
class Fibonacci
  include Enumerable

  def initialize(max)
    @max = max
  end

  def each
    return enum_for(:each) unless block_given?

    a, b = 1, 1
    while a <= @max
```

```

        yield a
        a, b = b, a + b
    end
end
end

# Uso
fib = Fibonacci.new(100)
fib.each { |n| print "#{n} " } # => 1 1 2 3 5 8 13 21 34 55 89

```

Exemplos Práticos

Processamento de Arquivos

```

def processar_arquivo(nome_arquivo)
  File.open(nome_arquivo) do |file|
    file.each_line do |linha|
      next if linha.strip.empty?
      yield linha if block_given?
    end
  end
end

processar_arquivo("dados.txt") do |linha|
  puts "Processando: #{linha}"
end

```

Transações em Banco de Dados

```

def with_transaction
  begin
    iniciar_transacao
    yield
    commit_transacao
  rescue
    rollback_transacao
  end
end

```

```

    raise
  end
end

with_transaction do
  # operações no banco de dados
  criar_usuario(dados)
  atualizar_perfil(perfil)
end

```

Boas Práticas

```

# Use &block para passar blocos como último parâmetro
def meu_metodo(opcoes = {}, &block)
  # configuração inicial
  block.call if block
end

# Verifique se um bloco foi fornecido
def metodo_com_bloco
  return "Nenhum bloco fornecido" unless block_given?
  yield
end

# Prefira map a each quando transformar dados
# Ruim
numeros = []
[1, 2, 3].each { |n| numeros << n * 2 }

# Melhor
numeros = [1, 2, 3].map { |n| n * 2 }

```

Padrões Comuns

Builder Pattern

```

class RelatorioBuilder
  def initialize
    @relatorio = Relatorio.new
  end

  def build
    yield(self)
    @relatorio
  end

  def adicionar_titulo(titulo)
    @relatorio.titulo = titulo
    self
  end

  def adicionar_conteudo(conteudo)
    @relatorio.conteudo = conteudo
    self
  end
end

relatorio = RelatorioBuilder.new.build do |b|
  b.adicionar_titulo("Relatório Mensal")
  .adicionar_conteudo("Dados do mês...")
end

```

See also

Documentação de Proc (<https://ruby-doc.org/core/Proc.html>)

Documentação do Enumerable (<https://ruby-doc.org/core/Enumerable.html>)

Conceitos Básicos de Blocos

Os blocos são uma das características mais poderosas e fundamentais do Ruby. Eles representam trechos de código que podem ser passados para métodos como argumentos.

Sintaxe de Blocos

Duas Formas de Escrever

```
# Usando chaves {} (recomendado para blocos de uma linha)
[1, 2, 3].each { |numero| puts numero }

# Usando do...end (recomendado para múltiplas linhas)
[1, 2, 3].each do |numero|
  dobro = numero * 2
  puts "O dobro de #{numero} é #{dobro}"
end
```

Parâmetros de Bloco

```
# Bloco sem parâmetros
3.times { puts "Olá!" }

# Bloco com um parâmetro
["a", "b", "c"].each { |letra| puts letra }

# Bloco com múltiplos parâmetros
hash = {nome: "Ruby", versao: "3.2"}
hash.each { |chave, valor| puts "#{chave}: #{valor}" }
```

Métodos que Aceitam Blocos

Yield


```

# Método simples com yield
def saudacao
  puts "Antes do bloco"
  yield
  puts "Depois do bloco"
end

saudacao { puts "Dentro do bloco!" }

# Yield com parâmetros
def repetir(quantidade)
  quantidade.times do |i|
    yield i
  end
end

repetir(3) { |n| puts "Repetição #{n}" }

```

Verificando Blocos

```

# Usando block_given?
def executar
  if block_given?
    puts "Executando o bloco:"
    yield
  else
    puts "Nenhum bloco fornecido"
  end
end

executar { puts "Olá!" }
executar # Sem bloco

```

Escopo e Variáveis

Variáveis Locais

```
# Blocos podem acessar variáveis externas
valor = 10
[1, 2, 3].each do |numero|
  soma = numero + valor
  puts soma
end

# Variáveis de bloco não vazam para fora
[1, 2, 3].each do |numero|
  temp = numero * 2
end
# puts temp # Isso geraria um erro
```

Variáveis de Bloco

```
# Parâmetros de bloco são locais ao bloco
total = 0
[1, 2, 3].each do |numero|
  total += numero
end
puts total # => 6
```

Exemplos Práticos

Manipulação de Arquivos

```
# Lendo um arquivo
def ler_arquivo(nome)
  File.open(nome, "r") do |arquivo|
    while linha = arquivo.gets
      yield linha if block_given?
    end
  end
end

ler_arquivo("dados.txt") do |linha|
```

```
puts "Lendo: #{linha.chomp}"  
end
```

Temporizador Simples

```
def medir_tempo  
  inicio = Time.now  
  yield  
  fim = Time.now  
  puts "Tempo decorrido: #{fim - inicio} segundos"  
end  
  
medir_tempo do  
  # código a ser medido  
  sleep(1)  
end
```

Boas Práticas

```
# Use {} para blocos de uma linha  
[1, 2, 3].map { |n| n * 2 }  
  
# Use do...end para blocos multilinhas  
[1, 2, 3].map do |n|  
  dobro = n * 2  
  dobro + 1  
end  
  
# Seja consistente com os nomes dos parâmetros  
["ruby", "python"].each { |linguagem| puts linguagem }  
  
# Evite blocos muito grandes  
# Prefira extrair para métodos quando o bloco ficar complexo  
def processar_dados(dados)  
  dados.map { |item| transformar(item) }  
end
```

```
def transformar(item)
  # lógica complexa aqui
end
```

Exercícios Comuns

```
# 1. Implementar um método que aceita um bloco
def repetir_mensagem
  yield if block_given?
end

# 2. Passar parâmetros para o bloco
def processar_lista(lista)
  lista.each { |item| yield item }
end

# 3. Combinar blocos com condicionais
def executar_se_valido(valor)
  if valor > 0
    yield valor
  else
    puts "Valor inválido"
  end
end
```

See also

Documentação de block_given? (https://ruby-doc.org/core/Kernel.html#method-i-block_given-3F)

Documentação de Proc (<https://ruby-doc.org/core/Proc.html>)

Iteradores em Ruby

Os iteradores são métodos que permitem percorrer coleções de elementos de forma elegante e eficiente.

Iteradores Básicos

each

```
# Iterando sobre um array
[1, 2, 3].each do |numero|
  puts "Número: #{numero}"
end

# Iterando sobre um hash
{nome: "Ruby", tipo: "Linguagem"}.each do |chave, valor|
  puts "#{chave}: #{valor}"
end
```

times

```
# Executando um bloco n vezes
5.times do |i|
  puts "Iteração #{i}"
end

# Versão compacta
3.times { puts "Ruby!" }
```

upto e downto

```
# Contagem crescente
1.upto(5) do |n|
  puts "Contando até #{n}"
end
```

```
# Contagem regressiva
5.downto(1) do |n|
  puts "Contagem regressiva: #{n}"
end
```

Iteradores de Transformação

map/collect

```
# Transformando elementos
numeros = [1, 2, 3, 4, 5]
dobrados = numeros.map { |n| n * 2 }
puts dobrados # => [2, 4, 6, 8, 10]

# Com múltiplas linhas
maiusculas = ["a", "b", "c"].map do |letra|
  letra.upcase
end
```

select/find_all e reject

```
# Filtrando elementos
numeros = (1..10).to_a
pares = numeros.select { |n| n.even? }
impares = numeros.reject { |n| n.even? }

# Com condições mais complexas
palavras = ["ruby", "python", "java", "javascript"]
longas = palavras.select do |palavra|
  palavra.length > 4
end
```

Iteradores Especializados

each_with_index

```
# Acesso ao índice durante iteração
frutas = ["maçã", "banana", "laranja"]
frutas.each_with_index do |fruta, indice|
  puts "#{indice + 1}. #{fruta}"
end
```

each_with_object

```
# Construindo um novo objeto durante iteração
resultado = ["a", "b", "c"].each_with_object({}) do |letra, hash|
  hash[letra] = letra.upcase
end
puts resultado # => {"a"=>"A", "b"=>"B", "c"=>"C"}
```

inject/reduce

```
# Acumulando valores
soma = [1, 2, 3, 4, 5].inject(0) { |acc, n| acc + n }
puts soma # => 15

# Exemplo com strings
palavras = ["ruby", "é", "incrível"]
frase = palavras.inject("") do |resultado, palavra|
  "#{resultado} #{palavra}".strip
end
```

Iteradores de Enumeráveis

any? e all?

```
# Verificando condições
numeros = [2, 4, 6, 8]
todos_pares = numeros.all? { |n| n.even? } # => true
algum_maior_que_cinco = numeros.any? { |n| n > 5 } # => true
```

find/detect

```
# Encontrando o primeiro elemento que satisfaz uma condição
primeiro_maior_que_cinco = (1..10).find { |n| n > 5 }
puts primeiro_maior_que_cinco # => 6
```

Exemplos Práticos

Processamento de Dados

```
class Produto
  attr_reader :nome, :preco

  def initialize(nome, preco)
    @nome = nome
    @preco = preco
  end
end

produtos = [
  Produto.new("Laptop", 5000),
  Produto.new("Mouse", 100),
  Produto.new("Teclado", 300)
]

# Filtrando produtos caros
caros = produtos.select { |p| p.preco > 1000 }

# Calculando valor total
total = produtos.inject(0) { |soma, p| soma + p.preco }
```

Manipulação de Arquivos

```
def processar_linhas(arquivo)
  File.readlines(arquivo).each_with_index do |linha, numero|
    yield linha.chomp, numero + 1 if block_given?
  end
end
```



```
processar_linhas("log.txt") do |conteudo, linha|
  puts "Linha #{linha}: #{conteudo}"
end
```

Boas Práticas

```
# Prefira map a each para transformações
# Ruim
resultados = []
numeros.each { |n| resultados << n * 2 }

# Melhor
resultados = numeros.map { |n| n * 2 }

# Use o iterador mais específico para sua necessidade
# Ruim
encontrado = nil
numeros.each do |n|
  if n > 5
    encontrado = n
    break
  end
end

# Melhor
encontrado = numeros.find { |n| n > 5 }
```

See also

Documentação do Enumerable (<https://ruby-doc.org/core/Enumerable.html>)

Documentação de Array (<https://ruby-doc.org/core/Array.html>)

Enumerators em Ruby

Enumerators são objetos que encapsulam a lógica de iteração, permitindo um controle mais fino sobre o processo de enumeração.

Conceitos Básicos

Criando Enumerators

```
# Usando to_enum
array = [1, 2, 3]
enum = array.to_enum
puts enum.next # => 1
puts enum.next # => 2

# Usando enum_for
hash = { a: 1, b: 2 }
enum = hash.enum_for(:each)
puts enum.next # => [:a, 1]
```

Enumerator como Iterador Externo

```
# Controle manual da iteração
letras = ['a', 'b', 'c'].to_enum
begin
  loop do
    puts letras.next
  end
rescue StopIteration
  puts "Iteração completa!"
end
```

Métodos Comuns

`each_with_index`

```
# Criando enumerator com índice
enum = %w[primeiro segundo terceiro].each_with_index
puts enum.next # => ["primeiro", 0]
puts enum.next # => ["segundo", 1]
```

with_index

```
# Personalizando o índice inicial
enum = %w[a b c].each
enum.with_index(1) do |letra, indice|
  puts "#{indice}: #{letra}"
end
```

Enumerators Especializados

Infinite Sequences

```
# Gerando sequência infinita
naturais = Enumerator.new do |y|
  n = 0
  loop do
    y << n
    n += 1
  end
end

puts naturais.take(5) # => [0, 1, 2, 3, 4]
```

Lazy Enumerators

```
# Processamento preguiçoso
infinitos = (1..Float::INFINITY).lazy
pares = infinitos.select(&:even?)
puts pares.take(5).force # => [2, 4, 6, 8, 10]
```

Casos de Uso Práticos

Processamento de Grandes Arquivos

```
def ler_arquivo_lazy(arquivo)
  Enumerator.new do |yielder|
    File.open(arquivo) do |file|
      file.each_line do |linha|
        yielder << linha.chomp
      end
    end
  end
end

leitor = ler_arquivo_lazy("dados.txt")
puts leitor.take(3) # Lê apenas as 3 primeiras linhas
```

Paginação

```
class Paginador
  def initialize(colecao, por_pagina)
    @colecao = colecao
    @por_pagina = por_pagina
  end

  def paginas
    Enumerator.new do |yielder|
      atual = @colecao
      until atual.empty?
        yielder << atual.take(@por_pagina)
        atual = atual.drop(@por_pagina)
      end
    end
  end
end

dados = (1..10).to_a
```

```
paginador = Paginador.new(dados, 3)
paginador.paginas.each { |pagina| p pagina }
```

Combinando Enumerators

Zip e Chain

```
# Combinando sequências
letras = %w[a b c].to_enum
numeros = [1, 2, 3].to_enum

combinados = letras.zip(numeros)
puts combinados.to_a # => [{"a", 1}, {"b", 2}, {"c", 3}]
```

Boas Práticas

```
# Use Enumerator quando precisar de controle fino
def processar_em_lotes(colecao, tamanho)
  colecao.each_slice(tamanho).with_index do |lote, indice|
    puts "Processando lote #{indice + 1}"
    lote.each { |item| yield item }
  end
end

# Use lazy para coleções potencialmente infinitas
def numeros_primos
  Enumerator.new do |yielder|
    n = 2
    loop do
      yielder << n if primo?(n)
      n += 1
    end
  end.lazy
end
```

See also

Documentação de Enumerator::Lazy (<https://ruby-doc.org/core/Enumerator/Lazy.html>)

Documentação de Enumerator (<https://ruby-doc.org/core/Enumerator.html>)

Closures em Ruby

Closures são blocos de código que podem capturar e carregar consigo o contexto onde foram definidos.

Conceitos Básicos

O que são Closures

```
# Exemplo básico de closure
def criar_contador
  count = 0
  return -> { count += 1 }
end

contador = criar_contador
puts contador.call # => 1
puts contador.call # => 2
```

Tipos de Closures em Ruby

```
# Blocks
def executar
  yield if block_given?
end

# Procs
soma = Proc.new { |a, b| a + b }

# Lambdas
multiplicar = ->(a, b) { a * b }
```

Diferenças entre Proc e Lambda

Verificação de Argumentos

```
# Lambda é mais estrito com argumentos
lambda_exemplo = ->(x, y) { x + y }
proc_exemplo = Proc.new { |x, y| x + y }

lambda_exemplo.call(1)      # ArgumentError
proc_exemplo.call(1)        # Retorna nil para y
```

Comportamento do return

```
def teste_lambda
  lambda = -> { return "lambda" }
  lambda.call
  return "método"
end

def teste_proc
  proc = Proc.new { return "proc" }
  proc.call
  return "método"
end

puts teste_lambda # => "método"
puts teste_proc   # => "proc"
```

Contexto e Escopo

Capturando Variáveis

```
def criar_saudacao(nome)
  saudacao = "Olá"
  -> { "#{saudacao}, #{nome}!" }
end

saudar = criar_saudacao("Ruby")
puts saudar.call # => "Olá, Ruby!"
```


Múltiplos Contextos

```
def criar_multiplicador(fator)
  -> (numero) { numero * fator }
end

dobrar = criar_multiplicador(2)
triplicar = criar_multiplicador(3)

puts dobrar.call(5)      # => 10
puts triplicar.call(5)   # => 15
```

Casos de Uso Práticos

Callbacks

```
class Button
  def initialize
    @callbacks = []
  end

  def on_click(&block)
    @callbacks << block
  end

  def click
    @callbacks.each { |callback| callback.call }
  end
end

botao = Button.new
botao.on_click { puts "Clicado!" }
botao.click   # => "Clicado!"
```

Memoização

```
def memoize
  cache = {}
  ->(arg) do
    unless cache.has_key?(arg)
      cache[arg] = yield(arg)
    end
    cache[arg]
  end
end

calcular = memoize { |n| n * 2 }
puts calcular.call(5) # Calcula
puts calcular.call(5) # Usa cache
```

Padrões de Design com Closures

Decorador

```
def decorar_logger(metodo)
  -> (*args) do
    puts "Chamando método com #{args}"
    resultado = metodo.call(*args)
    puts "Resultado: #{resultado}"
    resultado
  end
end

calculo = ->(x, y) { x + y }
com_log = decorar_logger(calculo)
com_log.call(2, 3)
```

Currying

```
def curry_exemplo
  ->(x) do
    ->(y) do
```

```

    ->(z) { x + y + z }
  end
end
end

soma = curry_exemplo
puts soma.call(1).call(2).call(3) # => 6

```

Boas Práticas

```

# Use lambdas para comportamentos reutilizáveis
validar = ->(valor) { valor.to_s.strip.length > 0 }
["", "ruby", " "].select(&validar)

# Evite closures muito complexos
# Prefira extrair para classes quando a lógica crescer
class Calculadora
  def initialize(operacao)
    @operacao = operacao
  end

  def calcular(x, y)
    @operacao.call(x, y)
  end
end

```

See also

Documentação de Proc (<https://ruby-doc.org/core/Proc.html>)

Documentação de Method (<https://ruby-doc.org/core/Method.html>)

Programação Orientada a Objetos em Ruby

Fundamentos

Em Ruby, tudo é um objeto. Esta não é apenas uma frase de efeito - é um princípio fundamental da linguagem.

O Básico dos Objetos

```
# Até números são objetos
42.class          # => Integer
42.methods.sort   # Lista todos os métodos disponíveis

# Strings também são objetos
"ruby".upcase     # => "RUBY"
"ruby".reverse    # => "ybur"
```

Os Quatro Pilares da OOP

1. Encapsulamento

```
class ContaBancaria
  def initialize(saldo_inicial)
    @saldo = saldo_inicial
  end

  def depositar(valor)
    @saldo += valor
  end

  private

  def validar_saldo
    @saldo > 0
  end
end
```

```
end  
end
```

2. Herança

```
class Animal  
  def falar  
    "Som genérico"  
  end  
end  
  
class Cachorro < Animal  
  def falar  
    "Au au!"  
  end  
end
```

3. Polimorfismo

```
class Ave  
  def voar  
    "Voando..."  
  end  
end  
  
class Pinguim < Ave  
  def voar  
    "Desculpe, não posso voar"  
  end  
end  
  
def fazer_voar(ave)  
  ave.voar  
end
```

4. Abstração

```

class Veiculo
  def iniciar
    ligar_motor
    verificar_sistemas
    liberar_freios
  end

  private

  def ligar_motor
    # Implementação específica
  end

  def verificar_sistemas
    # Implementação específica
  end
end

```

Características Únicas em Ruby

Duck Typing

```

def processar(objeto)
  if objeto.respond_to?(:calcular)
    objeto.calcular
  else
    "Objeto não suporta cálculo"
  end
end

```

Mixins com Modules

```

module Nadador
  def nadar
    "Nadando..."
  end
end

```

```
end

class Pato
  include Nadador
end

pato = Pato.new
pato.nadar # => "Nadando..."
```

Open Classes

```
class String
  def palindromo?
    self.downcase == self.downcase.reverse
  end
end

"ovo".palindromo? # => true
```

Boas Práticas

SOLID em Ruby

```
# Single Responsibility Principle
class RelatorioFinanceiro
  def gerar
    dados = coletar_dados
    formatar(dados)
  end

  private

  def coletar_dados
    # Lógica de coleta
  end
end
```

```
def formatar(dados)
  # Lógica de formatação
end
end
```

Composição vs Herança

```
# Preferir composição
class Carro
  def initialize
    @motor = Motor.new
    @transmissao = Transmissao.new
  end

  def ligar
    @motor.ligar
    @transmissao.engatar
  end
end
```

Padrões de Design Comuns

Factory Method

```
class DocumentoFactory
  def self.criar(tipo)
    case tipo
    when :pdf
      PDFDocument.new
    when :doc
      WordDocument.new
    else
      raise "Tipo de documento desconhecido"
    end
  end
end
```



```
end  
end
```

Singleton

```
require 'singleton'  
  
class Logger  
  include Singleton  
  
  def log(msg)  
    puts "[LOG] #{msg}"  
  end  
end  
  
Logger.instance.log("Mensagem")
```

Dicas e Truques

Method Missing

```
class Tradutor  
  def method_missing(nome_metodo, *args)  
    palavra = nome_metodo.to_s  
    if palavra.start_with?("traduzir_")  
      "Tradução de: #{palavra.sub('traduzir_', '')}"  
    else  
      super  
    end  
  end  
end
```

Reflexão

```
class Exemplo  
  def initialize
```

```

    @variavel = 42
  end

  def listar_metodos
    methods - Object.methods
  end

  def listar_variaveis
    instance_variables
  end
end

```

Recursos Avançados

Metaprogramação Básica

```

class Dinamico
  def self.criar_metodo(nome)
    define_method(nome) do |arg|
      "Método #{nome} chamado com #{arg}"
    end
  end
end

Dinamico.criar_metodo(:teste)

```

Delegação

```

require 'forwardable'

class Playlist
  extend Forwardable
  def_delegators :@songs, :<<, :first, :last

  def initialize
    @songs = []
  end
end

```

```
end  
end
```

See also

Documentação de Object (<https://ruby-doc.org/core/Object.html>)

Documentação de Module (<https://ruby-doc.org/core/Module.html>)

Documentação de Class (<https://ruby-doc.org/core/Class.html>)

Classes e Objetos em Ruby

Definindo Classes

Uma classe é um modelo para criar objetos. Em Ruby, classes são definidas usando a palavra-chave `class`.

```
class Pessoa
  def initialize(nome, idade)
    @nome = nome
    @idade = idade
  end

  def apresentar
    "Olá, me chamo #{@nome} e tenho #{@idade} anos"
  end
end

# Criando um objeto
pessoa = Pessoa.new("Ana", 25)
puts pessoa.apresentar # => "Olá, me chamo Ana e tenho 25 anos"
```

Atributos

Variáveis de Instância

```
class Conta
  def initialize(saldo)
    @saldo = saldo # Variável de instância
  end
end
```

Getters e Setters

```
class Pessoa
  # Forma manual
```

```

def nome
  @nome
end

def nome=(novo_nome)
  @nome = novo_nome
end

# Forma automática usando attr_*
attr_reader :idade      # Getter
attr_writer :email      # Setter
attr_accessor :telefone # Getter e Setter
end

```

Métodos

Métodos de Instância

```

class Calculadora
  def somar(a, b)
    a + b
  end

  def subtrair(a, b)
    a - b
  end
end

calc = Calculadora.new
puts calc.somar(5, 3) # => 8

```

Métodos de Classe

```

class Tempo
  def self.agora
    Time.now
  end
end

```

```

end

class << self
  def amanha
    Time.now + 86400
  end
end

puts Tempo.agora      # Imprime o tempo atual
puts Tempo.amanha     # Imprime o tempo de amanhã

```

Construtores

```

class Produto
  def initialize(nome, preco)
    @nome = nome
    @preco = preco
    @criado_em = Time.now
  end

  # Construtor alternativo
  def self.from_hash(hash)
    new(hash[:nome], hash[:preco])
  end
end

# Diferentes formas de criar objetos
produto1 = Produto.new("Livro", 29.90)
produto2 = Produto.from_hash({ nome: "Caneta", preco: 4.50 })

```

Visibilidade de Métodos

```

class Documento
  def publico

```

```

    "Qualquer um pode me chamar"
  end

  protected

  def protegido
    "Apenas classes relacionadas podem me chamar"
  end

  private

  def privado
    "Apenas métodos internos podem me chamar"
  end
end

```

Métodos Predicados

```

class Pessoa
  def initialize(idade)
    @idade = idade
  end

  def maior_de_idade?
    @idade >= 18
  end

  def aposentado?
    @idade >= 65
  end
end

pessoa = Pessoa.new(30)
puts pessoa.maior_de_idade? # => true
puts pessoa.aposentado?    # => false

```

Comparação de Objetos

```
class Ponto
  attr_reader :x, :y

  def initialize(x, y)
    @x = x
    @y = y
  end

  def ==(outro)
    x == outro.x && y == outro.y
  end

  def eql?(outro)
    self == outro
  end

  def hash
    [x, y].hash
  end
end
```

Objetos Singleton

```
class Configuracao
  @instancia = nil

  def self.instancia
    @instancia ||= new
  end

  private_class_method :new
end
```


Métodos Especiais

```
class Pessoa
  def initialize(nome)
    @nome = nome
  end

  def to_s
    "Pessoa: #{@nome}"
  end

  def inspect
    "<Pessoa:#{object_id} @nome='#{@nome}'>"
  end

  def respond_to_missing?(metodo, include_private = false)
    metodo.to_s.start_with?('falar_') || super
  end

  def method_missing(metodo, *args)
    if metodo.to_s.start_with?('falar_')
      "#{@nome} está falando #{metodo.to_s.sub('falar_', '')}"
    else
      super
    end
  end
end
```

Boas Práticas

```
class Usuario
  # Use attr_reader ao invés de attr_accessor quando possível
  attr_reader :nome, :email

  # Valide dados no initialize
  def initialize(nome, email)
```

```

    raise ArgumentError, "Nome não pode ser vazio" if nome.nil? ||
nome.empty?
    raise ArgumentError, "Email inválido" unless email.include?
('@')

    @nome = nome
    @email = email
end

# Prefira métodos pequenos e focados
def nome_completo
    "#{@nome} #{@sobrenome}"
end

# Use métodos predicados apropriadamente
def valido?
    nome_valido? && email_valido?
end

private

def nome_valido?
    @nome.length >= 2
end

def email_valido?
    @email.include?('@@') && @email.include?('.')
end
end

```

See also

Documentação de Object (<https://ruby-doc.org/core/Object.html>)

Documentação de Class (<https://ruby-doc.org/core/Class.html>)

Herança em Ruby

Conceitos Básicos

A herança é um dos pilares fundamentais da programação orientada a objetos. Em Ruby, uma classe pode herdar características de apenas uma classe pai (herança simples).

```
class Animal
  def initialize(nome)
    @nome = nome
  end

  def falar
    "Som genérico"
  end
end

class Cachorro < Animal
  def falar
    "Au au!"
  end
end

rex = Cachorro.new("Rex")
puts rex.falar # => "Au au!"
```

Super

Usando super com initialize

```
class Veiculo
  def initialize(marca)
    @marca = marca
  end
end
```

```
class Carro < Veiculo
  def initialize(marca, modelo)
    super(marca)
    @modelo = modelo
  end
end

fusca = Carro.new("VW", "Fusca")
```

Super sem Parênteses

```
class Animal
  def dormir
    "ZzZzZz..."
  end
end

class Gato < Animal
  def dormir
    super + " Miau..."
  end
end
```

Hierarquia de Classes

Verificando a Hierarquia

```
class A; end
class B < A; end
class C < B; end

puts C.ancestors # => [C, B, A, Object, Kernel, BasicObject]
puts C.superclass # => B
```

is_a? e kind_of?

```
gato = Gato.new
puts gato.is_a?(Gato)      # => true
puts gato.is_a?(Animal)   # => true
puts gato.kind_of?(Object) # => true
```

Métodos e Visibilidade

Sobrescrevendo Métodos

```
class Pessoa
  def apresentar
    "Olá, sou uma pessoa"
  end
end

class Funcionario < Pessoa
  def apresentar
    "#{super} e trabalho aqui"
  end
end
```

Visibilidade na Herança

```
class Animal
  protected
  def protegido
    "Método protegido"
  end

  private
  def privado
    "Método privado"
  end
end

class Cachorro < Animal
```

```

def chamar_protegido
  protegido # OK
end

def chamar_privado
  privado # OK
end
end

```

Herança e Composição

Quando Usar Cada Uma

```

# Herança - "é um"
class Ave
  def voar
    "Voando..."
  end
end

class Papagaio < Ave
  def falar
    "Olá!"
  end
end

# Composição - "tem um"
class Motor
  def ligar
    "Motor ligado"
  end
end

class Carro
  def initialize
    @motor = Motor.new
  end
end

```

```
def ligar
  @motor.ligar
end
end
```

Abstract e Interface

Simulando Classes Abstratas

```
class Animal
  def falar
    raise NotImplementedError, "#{self.class} precisa implementar
o método 'falar'"
  end
end

class Cachorro < Animal
  def falar
    "Au au!"
  end
end
```

Simulando Interfaces

```
module Nadador
  def nadar
    raise NotImplementedError
  end
end

class Peixe
  include Nadador

  def nadar
    "Nadando como um peixe"
  end
end
```



```
end  
end
```

Boas Práticas

Princípio de Substituição de Liskov

```
class Retangulo  
  def initialize(largura, altura)  
    @largura = largura  
    @altura = altura  
  end  
  
  def area  
    @largura * @altura  
  end  
end  
  
class Quadrado < Retangulo  
  def initialize(lado)  
    super(lado, lado)  
  end  
end
```

Evitando Herança Profunda

```
# Evite isso  
class A; end  
class B < A; end  
class C < B; end  
class D < C; end # Muito profundo!  
  
# Prefira composição  
class D  
  def initialize  
    @b = B.new
```

```
@c = C.new  
end  
end
```

Padrões Comuns

Template Method

```
class Relatorio  
  def gerar  
    coletar_dados  
    processar_dados  
    formatar_saida  
  end  
  
  private  
  
  def coletar_dados  
    raise NotImplementedError  
  end  
  
  def processar_dados  
    raise NotImplementedError  
  end  
  
  def formatar_saida  
    raise NotImplementedError  
  end  
end  
  
class RelatorioVendas < Relatorio  
  private  
  
  def coletar_dados  
    # Implementação específica  
  end  
end
```

```
def processar_dados
  # Implementação específica
end

def formatar_saida
  # Implementação específica
end
end
```

Factory Method

```
class Animal
  def self.criar(tipo)
    case tipo
    when :cachorro
      Cachorro.new
    when :gato
      Gato.new
    else
      raise "Tipo de animal desconhecido"
    end
  end
end
```

See also

Documentação de Class#superclass (<https://ruby-doc.org/core/Class.html#method-i-superclass>)

Documentação de Module#ancestors (<https://ruby-doc.org/core/Module.html#method-i-ancestors>)

Encapsulamento em Ruby

Conceitos Básicos

O encapsulamento é um dos princípios fundamentais da programação orientada a objetos, que consiste em esconder os detalhes internos de implementação e fornecer uma interface pública para interação com o objeto.

Variáveis de Instância

```
class ContaBancaria
  def initialize(saldo_inicial)
    @saldo = saldo_inicial # Variável de instância privada
  end

  def consultar_saldo
    @saldo
  end

  def depositar(valor)
    @saldo += valor if valor > 0
  end
end

conta = ContaBancaria.new(1000)
puts conta.consultar_saldo # => 1000
```

Níveis de Acesso

Public

```
class Produto
  def initialize(nome, preco)
    @nome = nome
    @preco = preco
  end
end
```

```
# Métodos públicos por padrão
def descricao
  "#{@nome} - R$ #{@preco}"
end
end
```

Protected

```
class Funcionario
  def initialize(salario)
    @salario = salario
  end

  def compara_salario(outro_funcionario)
    @salario > outro_funcionario.salario_protegido
  end

  protected

  def salario_protegido
    @salario
  end
end
```

Private

```
class Processador
  def executar
    preparar
    processar
    finalizar
  end

  private
```

```
def preparar
  # Lógica interna
end

def processar
  # Lógica interna
end

def finalizar
  # Lógica interna
end
end
```

Getters e Setters

Métodos Tradicionais

```
class Pessoa
  def initialize(nome)
    @nome = nome
  end

  # Getter
  def nome
    @nome
  end

  # Setter
  def nome=(novo_nome)
    @nome = novo_nome
  end
end
```

Usando attr_*

```

class Produto
  # Getter e Setter
  attr_accessor :nome, :preco

  # Apenas Getter
  attr_reader :codigo

  # Apenas Setter
  attr_writer :desconto

  def initialize(nome, preco, codigo)
    @nome = nome
    @preco = preco
    @codigo = codigo
    @desconto = 0
  end
end

```

Validações e Controle de Acesso

Validando Dados

```

class Conta
  def initialize
    @saldo = 0
  end

  def depositar(valor)
    raise ArgumentError, "Valor deve ser positivo" unless valor >
0
    @saldo += valor
  end

  def sacar(valor)
    raise ArgumentError, "Saldo insuficiente" if valor > @saldo
    @saldo -= valor
  end
end

```

```
end  
end
```

Customizando Getters e Setters

```
class Produto  
  def preco  
    @preco  
  end  
  
  def preco=(novo_preco)  
    raise "Preço inválido" if novo_preco < 0  
    @preco = novo_preco  
  end  
  
  def desconto=(valor)  
    @desconto = if valor > 0.3  
                  0.3 # Limita desconto a 30%  
                else  
                  valor  
                end  
  end  
  
end  
end
```

Padrões de Encapsulamento

Value Object

```
class Coordenada  
  attr_reader :x, :y  
  
  def initialize(x, y)  
    @x = x  
    @y = y  
  end  
end
```



```

# Objetos imutáveis
def mover(dx, dy)
  Coordenada.new(@x + dx, @y + dy)
end
end

```

Tell, Don't Ask

```

# Evite isso
class Pedido
  def calcular_desconto
    if @cliente.vip?
      @valor * 0.2
    else
      @valor * 0.1
    end
  end
end

# Prefira isso
class Pedido
  def calcular_desconto
    @cliente.aplicar_desconto(@valor)
  end
end

class Cliente
  def aplicar_desconto(valor)
    desconto = vip? ? 0.2 : 0.1
    valor * desconto
  end
end

```

Boas Práticas

Princípio da Responsabilidade Única

```

# Ruim
class Usuario
  def salvar
    # Lógica de banco de dados
  end

  def enviar_email
    # Lógica de envio de email
  end
end

# Melhor
class Usuario
  def salvar
    UsuarioRepository.new.salvar(self)
  end
end

class NotificadorEmail
  def notificar(usuario)
    # Lógica de envio de email
  end
end

```

Lei de Demeter

```

# Evite isso
class Pedido
  def processar
    @cliente.carteira.conta.debitar(@valor)
  end
end

# Prefira isso
class Pedido
  def processar
    @cliente.debitar(@valor)
  end
end

```

```
    end
  end

  class Cliente
    def debitar(valor)
      @carteira.debitar(valor)
    end
  end
end
```

See also

Documentação de attr_accessor (https://ruby-doc.org/core/Module.html#method-i-attr_accessor)

Documentação de visibilidade de métodos (<https://ruby-doc.org/core/Module.html#method-i-private>)

Composição em Ruby

Conceitos Básicos

A composição é um princípio de design onde objetos complexos são construídos a partir de objetos mais simples, estabelecendo uma relação "tem um" entre eles.

Composição vs Herança

```
# Herança - "é um"
class Ave
  def voar
    "Voando..."
  end
end

class Papagaio < Ave
end

# Composição - "tem um"
class Aviao
  def initialize
    @motor = Motor.new
    @asas = Asas.new
  end

  def voar
    return "Não é possível voar" unless @motor.funcionando? &&
    @asas.ok?
    "Decolando..."
  end
end
```

Implementando Composição

Composição Básica

```

class Carro
  def initialize
    @motor = Motor.new
    @transmissao = Transmissao.new
    @rodas = Array.new(4) { Roda.new }
  end

  def ligar
    @motor.ligar
    @transmissao.em_neutro
  end

  def dirigir
    return false unless @motor.ligado?
    @transmissao.engatar_primeira
    true
  end
end
end

```

Injeção de Dependências

```

class Computador
  def initialize(processador:, memoria:, armazenamento:)
    @processador = processador
    @memoria = memoria
    @armazenamento = armazenamento
  end

  def iniciar
    @processador.ligar
    @memoria.verificar
    @armazenamento.montar
  end
end

pc = Computador.new(
  processador: Intel.new,

```

```
memoria: RAM.new(16),  
armazenamento: SSD.new(512)  
)
```

Padrões de Composição

Delegação

```
class Playlist  
  def initialize  
    @musicas = []  
    @player = AudioPlayer.new  
  end  
  
  # Delegando métodos para @player  
  def play  
    @player.play(@musicas.first)  
  end  
  
  def pause  
    @player.pause  
  end  
  
  def add_musica(musica)  
    @musicas << musica  
  end  
end
```

Composição com Módulos

```
module Logger  
  def log(mensagem)  
    puts "[#{Time.now}] #{mensagem}"  
  end  
end
```

```

class ServicoEmail
  def initialize
    @logger = Object.new.extend(Logger)
  end

  def enviar(email)
    @logger.log("Enviando email para #{email}")
    # Lógica de envio
  end
end

```

Composição Flexível

Componentes Intercambiáveis

```

class Notificador
  def initialize(servicos = [])
    @servicos = servicos
  end

  def notificar(mensagem)
    @servicos.each { |servico| servico.enviar(mensagem) }
  end
end

notificador = Notificador.new([
  EmailNotificacao.new,
  SMSNotificacao.new,
  SlackNotificacao.new
])

```

Composição Dinâmica

```

class Editor
  def initialize
    @plugins = {}
  end
end

```

```

end

def adicionar_plugin(nome, plugin)
  @plugins[nome] = plugin
end

def remover_plugin(nome)
  @plugins.delete(nome)
end

def executar_plugin(nome, *args)
  if @plugins[nome]
    @plugins[nome].executar(*args)
  end
end
end

```

Boas Práticas

Composição sobre Herança

```

# Evite herança profunda
class Animal < Servivo < Organismo < Entidade # Ruim

# Prefira composição
class Animal
  def initialize
    @metabolismo = Metabolismo.new
    @reproducao = Reproducao.new
    @movimento = Movimento.new
  end
end

```

Interfaces Claras


```

class Relatorio
  def initialize(formatador:, dados:)
    @formatador = formatador
    @dados = dados
  end

  def gerar
    conteudo = processar_dados
    @formatador.formatar(conteudo)
  end

  private

  def processar_dados
    @dados.map { |dado| dado.to_h }
  end
end

```

Exemplos Práticos

Sistema de Pagamento

```

class ProcessadorPagamento
  def initialize(gateway:, validador:, notificador:)
    @gateway = gateway
    @validador = validador
    @notificador = notificador
  end

  def processar(pagamento)
    return false unless @validador.validar(pagamento)

    if @gateway.cobrar(pagamento)
      @notificador.sucesso(pagamento)
      true
    else

```

```
        @notificador.falha(pagamento)
        false
    end
end
end
```

Gerador de Relatórios

```
class GeradorRelatorio
  def initialize
    @coletores = []
    @processadores = []
    @formatadores = []
  end

  def adicionar_coletor(coletor)
    @coletores << coletor
  end

  def adicionar_processador(processador)
    @processadores << processador
  end

  def adicionar_formatador(formatador)
    @formatadores << formatador
  end

  def gerar
    dados = @coletores.map(&:coletar).flatten
    dados = @processadores.reduce(dados) { |acc, proc|
      proc.processar(acc) }
    @formatadores.map { |fmt| fmt.formatar(dados) }
  end
end
```

See also

Documentação de Modules (<https://ruby-doc.org/core/Module.html>)

Documentação de Object (<https://ruby-doc.org/core/Object.html>)